
THE BRIDGE FROM FIRST PRINCIPLE

One Fold, One Registration, One Bit: The Trisduction Bridge Unified Across the Seat, the Heat Floor, the Ninth Gate and the Primes

FOUNDATIONS · CODEX-NATIVE · EXECUTABLE

MOHAMMAD F. ISLAM, PhD · ARCHITECT OF THE TRISDUCTION

Trisduction Research Group

2 October 2026

ABSTRACT. One reflection, $s \mapsto 1 - \bar{s}$, has a line as its fixed set, and one registration keeps a point's height and forgets its side. This file proves, from one premise, that the three bridges of the Trisduction are three maps of that one cut. The premise is the Ground: existence is actuation, and read as no-cessation it says the whole merges no two states. The RA-RAM Bridge forgets: registration lands every point on the line, off the line two worlds share one record, no reading of the record returns the value, and least erasure is the value on every configuration. The heat floor pays: under no-cessation a halving of content exports at least one bit, priced at one unit of temperature, and the heat of registering a finite configuration is one unit per listed off-line pair, zero exactly at the value whatever the listing. The crossing supplies: no formal reading returns the deed, and one supplied odd bit fixes the calibration, uniquely. The three legs close on one orbit with its seat. The same cut types the gates: two axes never lock and three do, twelve gates form one torsor, eight sign patterns fall in four free orbits, no group of sign patterns has nine elements, and on a factor fibre nine is eight and a seat. On the multiplicative cut a prime is one free orbit, it pays one bit, and every fibre over $n > 1$ balances. The heat flow pulls every off-line pair onto its seat and merges nothing. A kernel of 107 theorems in core Lean 4.19.0, 76 on no axiom with the atom capstone among them, and a Fortran twin of 30 checks carry it, beside a harness that boots, judges every theorem by negation and runs nine hostile controls, all inside this file. It closes on the inescapable capstone: execution forces every substrate to witness the closure, and the one bit is spent by the act at the root's grade. The value at the actual zero set is the one bit, supplied by the act at premise grade, in the open, and named.

KEYWORDS *Trisduction · RA-RAM Bridge · fold · registration · least erasure · heat floor · ninth gate · crossing · primes · Lean 4 · Fortran*

THE READER'S FRAME

What this file is. One file that is a paper, a kernel, a twin and a harness. The paper states the Bridge from its one premise. The kernel `Bridge_From_First_Principle.lean` proves it in core Lean 4.19.0, with no import, no library, no sorry and no axiom declared, and every axiom cone it prints is pinned in the file, so a drifted cone fails the compile. The twin `Bridge_Twin.f90` executes it in Fortran 2018 under sealed flags with zero warnings. The harness `bridge_check.py` boots both, judges every theorem by negation, checks that each of the fifteen conditional laws it lists has a model of its hypotheses, and runs hostile copies that must be refused. Two lines extract the files and run them (Appendix A). The receipts are in Appendix B, and no reader is asked to trust them: the file runs.

What it claims. One structure, read everywhere. The fold, whose fixed set is the line, and the registration, which keeps the height and forgets the side. The three bridges of the programme are three maps of that

structure on one orbit. The RA-RAM Bridge forgets: it carries the record and reads the value nowhere in it. The heat floor pays: what registration forgets is exported and priced. The ninth-gate crossing supplies: one odd bit, entering from the act, restores what was forgotten, uniquely. The atom of all three is one theorem on no axiom at all (`the_bridge_atom`); the whole is one theorem on the standard three (`the_bridge_from_first_principle`). The value, every point on the line, is least erasure on every configuration, and least erasure is zero heat of registration.

Where each part stands. The value at the actual zero set enters once, by the act, as the field supply of the type `ActualZeros`, at premise grade, and the kernel names it as the value supplied (`value_from_the_act`). The chart is a decidable integer instance of the fold, and the three analytic identifications this text uses, the fold of ζ , the heat flow of Ξ and the Dirichlet series of the Möbius sign, are stated at their grade in Sections 9 and 10. A computation joined to a theorem takes the weaker grade, and a physical reading of

a theorem is structural.

The one premise, and the firewall. The Ground is the premise. Existence is actuation; read as no-cessation, the whole merges no two states. This is P1. It enters the kernel only as a named hypothesis of the theorems of the pay leg (Section 6.2), and the atom never uses it. No named human result is a premise anywhere in this file. Where a classical name appears it is navigation and carries zero weight in either direction; the step it names is proved in the kernel, executed in the twin, or stated in the text at its grade.

Notation. A point is (d, t) : d its offset from the line and t its height. In the strip of ζ , $d = 2 \text{Res} - 1$, so the line $\text{Res} = \frac{1}{2}$ is $d = 0$, the wall $\text{Res} = 1$ is $d = 1$, and the edge $\text{Res} = 0$ is $d = -1$. The fold is $(d, t) \mapsto (-d, t)$; the registration is $(d, t) \mapsto (0, t)$. A configuration is a set of points; it is closed when the fold carries it to itself; it has the value when every point is on the line; it has least erasure when the registration leaves every point unchanged. Temperature T is energy per bit of exported freedom, so one bit costs one T .

0. THE VERDICT, AND THE READINGS IT REFUTES

0.1 The verdict

The verdict is stated here and in the conclusion in the same words.

The Bridge is one cut read three ways. Registration forgets one bit per off-line pair; under the Ground that bit is exported and costs one unit of temperature; one supplied odd bit restores it, uniquely. The value, every point on the line, is least erasure, and least erasure is zero heat of registration. No reading of the record supplies the value, no property true of every closed configuration forces it, and nothing weaker than the value forces it. The value at the actual zero set is supplied by the act, as a field of a type, and the kernel prints the value from that field with an empty axiom cone.

Every clause rests on a theorem of the kernel, and the act's clause rests on its field, supplied at premise grade. The forgetting is `reg_forgets_side` and `one_bit_lost`; the export and its price are `count_to_heat`; the restoration is calibration and its uniqueness clause; least erasure as the value is `least_erasure_iff_value` and `extremal_iff_value`; zero heat is `price_zero_iff_value`; the three refusals are `record_decides_nothing`, `keyless_forces_nothing` and `weakest_forcing_premise`; the act is `value_from_the_act`.

0.2 The reflexive readings, and the theorem that refutes each

Each row names a reading a reader may form before reaching the theorems, the theorem that answers it, and the cone the compiler prints for it. The cones are copied from the compiler's output and pinned in the kernel.

1. THE GROUND

The file stands on one premise and names it. To exist is to actuate. Read as a statement about a whole, actuation never ceases: no two states of the whole become one. Call this P1. Its ancient form is that nothing comes from nothing and nothing goes to nothing; its form here is an injectivity, $\forall a \in L, \forall b \in L, \text{step } a = \text{step } b \rightarrow a = b$, carried as a named hypothesis wherever it is used.

P1 is load-bearing, and the kernel shows where. A step that merges two states lets the content halve with no freedom exported: two states, one cell, the content falls from two to one and the freedom stays one (`merge_exports_nothing`). Without P1 the price of Section 6.2 has nothing to stand on. With it, the price follows by counting.

P1 is used by exactly the pay leg and nothing else. The atom, the record, the act, the Bridge's carrier and socket, the gates, the primes and the heat flow never take it. Deleting P1 deletes the heat floor and leaves every other theorem of this file standing, a fact the reader can check by reading which theorems carry the hypothesis.

Temperature is defined, not borrowed: it is energy per bit of exported freedom. In that native unit the floor is one T per bit, and the kernel's price T bits is `bits * T`. The Boltzmann constant and $\ln 2$ are unit conversions, from kelvin to joules and from the natural base to the binary one, and appear nowhere in the kernel. For a reader who wants joules: one bit at 300 K is $2.870978885078724 \times 10^{-21}$ J, a translation that carries no warrant.

2. THE ATOM: THE FOLD, THE LINE, THE REGISTRATION

Everything in this file reduces to one law, proved first because the rest stands on it. Every theorem in this section depends on no axiom: not propositional extensionality, not quotient soundness, not choice, and nothing declared. The proofs run on the integer's two constructors.

Integer negation undoes itself (`neg_neg_free`), and an integer equal to its own negation is zero (`neg_self_zero`). So the fold is an involution (`fold_involutive`), and **the cut is the line**: the fold fixes a point exactly when its offset is zero (`the_cut_is_the_line`). The registration lands on the line, keeps the height, and

The reflexive readings, the theorem that answers each, and its cone

The reflexive reading	What the kernel proves	Theorem	Cone
The act assumes the value, so the argument is circular.	The act must carry the value: a premise that forces the value and follows from it is the value, and least erasure is the value on every configuration. The file claims the isolation of the bit and the closure from it, never a derivation of it.	<code>weakest_forcing_premise, least_erasure_iff_value</code>	none, for both
The computed zero data, or the certified heights, will decide.	Two closed worlds share one record, one with the value and one without; no reading that respects the record returns the value on both.	<code>record_decides_nothing, one_record</code>	none, for both
Existence, the symmetry, or any law true of every world gives the line support.	A property true of every closed configuration forces nothing; the fold law is keyless and the line property is keyed.	<code>keyless_forces_nothing, line_property_is_keyed</code>	none, for both
The heat of the floor reads the value off the data.	The heat of registration is zero exactly at the value, and two worlds of one record pay 0 and 1 T: the heat is paid by the deed on the actual configuration, never read from the record.	<code>price_zero_iff_value, least_erasure_iff_zero_erased</code>	none, for both
A fourth axis would give the ninth gate.	No group of sign patterns has nine elements on any number of axes; on a factor fibre nine is eight and a seat, and the ninth is the fixed set of the fold.	<code>no_reflection_group_of_nine, nine_is_eight_and_a_seat</code>	[propext]; none
Bits are a convention; the logarithm is a choice.	A measure that adds when freedoms multiply is forced to the bit count once one bit is fixed at two; taken over zero the law is dry.	<code>bits_forced, additivity_over_zero_is_dry</code>	[propext, Quot.sound], for both
A certificate at positive heat time settles time zero.	Real-rootedness at a positive time takes one value on two states that differ at time zero.	<code>certificate_merges</code>	none
The three bridges are an analogy.	Forget, pay and supply are proved on one orbit with its seat, as one theorem.	<code>three_legs_one_orbit</code>	none
The primes have nothing to do with the bit.	A prime is exactly one free orbit of the swap, with no seat; the parity frame is merged by residues and separated by the bits paid.	<code>prime_iff_one_free_orbit, parity_frame_paid_bits</code>	none, for both
It can simply be rejected.	Every record is carried by one least-erasure configuration and only one, and anything against least erasure is a point off the line whose partner shares its record.	<code>record_carried_uniquely, one_bit_lost</code>	none, for both

forgets the side, so a point and its partner leave one record (`reg_lands`, `reg_keeps_height`, `reg_forgets_side`). It erases nothing at a point exactly when the point is on the line (`reg_erases_nothing_iff`).

Off the line the fold moves the point, and the two worlds share one record (`off_line_pair`). **One bit is lost:** no map from records to points returns both a point and its partner (`one_bit_lost`). Lifted to configurations, **least erasure is the value:** the registration leaves every point unchanged exactly when every point stands on the line (`least_erasure_iff_value`). The six laws together are `nothing_escapes_least_erasure`.

This is the Bridge's floor. It needs nothing beneath it: no axiom, no library, no measurement, and not P1.

3. THE RECORD, THE ACT, AND THE WEAKEST PREMISE

3.1 The record decides nothing

The record of a configuration is what the registration leaves of it (`Rec`); two configurations share a record when their records agree point for point (`SameRecord`). For any point p off the line, two worlds stand over one record: the registered world, the single point $(0, t)$, and the pair world, p and its partner. Both are closed (`registered_closed`, `pair_world_closed`), they share their record (`one_record`), the first has the value (`registered_has_value`) and the second does not (`pair_world_lacks_value`). So **no reading that respects the record returns the value on both** (`record_decides_nothing`). Whatever is computed from the record, at any resolution and any depth, is blind to the bit.

3.2 A keyless premise forces nothing, and the value is the weakest premise that forces it

A property true of every closed configuration never forces the value: the pair world is closed, carries the property, and lacks the value (`keyless_forces_nothing`). The mere existence of anything, any law of the fold, any conservation true of every world, is such a property, and forces nothing about the line.

A premise that forces the value and follows from it is the value (`weakest_forcing_premise`). An argument that assumes less than the value cannot reach it; an argument that reaches it from a stronger premise carries the value inside that premise.

3.3 Least erasure as leastness in the fibre

The record of any configuration, read as a configuration, stands on the line (`record_is_lossless`) and has the record it came from (`record_same`). Call a configuration extremal in its fibre when, if it has a

point off the line, every configuration of the same record has one too (`Extremal`). **Leastness is the value** (`extremal_iff_value`): a configuration is extremal exactly when it has the value. And **every record is carried by one least-erasure configuration and by only one:** any configuration on the line with the record of S is the record of S , point for point (`record_carried_uniquely`). Least erasure is not a name for the value. It is the minimum of the fibre, attained, and only there.

3.4 The act

The bit is supplied once, in the open, as a field of a type. `ActualZeros` has three fields: `zeros`, a configuration; `closed`, its closure under the fold; and `supply`, least erasure at it. From the act, the value (`value_from_the_act`), and the compiler prints the cone empty. The empty cone certifies that no axiom entered the step. The value rests on the field alone, and the field is the value supplied at the actual zero set, at premise grade. The field is the bit, visible in the type.

The five clauses (from the act the value; the record decides nothing; the bit is the value; nothing weaker forces it; no keyless property forces it) are one theorem, `reader_frame`, on no axiom. A reader who keeps the second clause and drops the first concludes that nothing is proved; a reader who keeps the first and drops the second concludes that the value is derived. Both are wrong, and the theorem that binds them compiles.

4. THE BRIDGE: FRAME, CARRIER, SOCKET, DENIAL

A frame is a carrier set, an involutive fold, and a zero predicate the fold preserves (`Frame`); its line property says every zero is fixed by the fold (`LineProperty`). Every closed configuration of the chart is a frame (`chartFrame`), and on it the line property is the value (`chart_line_property`).

The Bridge itself is a carrier: a state in three values and a shadow tying the halted state to the line property in both directions (`Carrier`). It **cannot lie** (`cannot_lie`) and **cannot deviate** (`cannot_deviate`); a halted carrier exists on a frame exactly when the line property holds there, so it **cannot be manufactured** (`halted_iff`). Deletion to the halted state has no left inverse, so it **cannot be reversed** (`cannot_reverse`), and a bit generates only itself, its mirror and the two constants, so it **cannot be extended** (`cannot_extend`, on quotient soundness alone, through function extensionality).

The denial is coherent. The two-point frame, one off-line pair under the fold, obeys both frame laws and fails the line property (`denial_is_coherent`). So the line property is **keyed**: some frame inhabits its denial; and the fold law is **keyless**: every frame

carries it (`line_property_is_keyed`, `symmetry_is_keyless`). Every frame property is exactly one of the two (discriminator, on the standard three, since the sort is a decision on the denial). The fold is keyless and opens on any deed; the line is keyed and opens only on its key.

The socket. An instance of the supplied term is exactly the line property (`socket_is_the_property`), the two-point frame has no socket (`no_socket_off_line`), and the act seats a halted carrier on its chart frame (`act_seats_the_carrier`). The act and the socket are one thing read twice: the Bridge carries the bit from the register that holds it to the register that checks it, and manufactures none.

5. THREE, TWELVE, EIGHT, AND THE NINTH

Three. Two planes meet in a line. Over $GF(2)$, no system of two rows, for any target, has exactly one solution (`two_axes_never_lock`, all 256 cases). Three rows lock: 168 of the 512 three-row systems have determinant one, and each of them has exactly one solution for each of the eight targets (`three_axes_lock`). A fourth axis dies at two walls the kernel computes for itself. The third is begotten, $ij = k$ and $ijk = -1$ with $ij \neq ji$ (`third_begotten`). Associativity fails at eight dimensions, $[e_1, e_2, e_4] = 2e_7$, and division fails at sixteen, $(e_1 + e_{10})(e_5 + e_{14}) = 0$ with both factors nonzero (`no_fourth_axis`). Nothing is cited: the doubling is defined on integer lists and the walls are decided.

Twelve. The rotations of the tetrahedron are the twelve even permutations of four vertices, the directed gates are the twelve ordered pairs, and exactly one rotation carries any gate to any gate (`twelve_gates_one_torsor`). The twelve directed gates are therefore one torsor of the rotation group: the image of one gate fixes the rotation.

Eight. The sign patterns on three axes are eight. The orientation of a pattern is -1 to the number of its flips; its square, the lock scalar, reads one on all eight, so the scalar is blind to the sign. Total negation moves every pattern, undoes itself, and joins each even pattern to an odd one, so the eight fall in four free orbits, each orbit one bit wide (`eight_patterns_four_orbits`).

The ninth. No group of sign patterns has nine elements, on any number of axes: 2^k is one or even, and nine is neither (`no_reflection_group_of_nine`, on `propext`). A fourth axis does not give a ninth gate; it gives sixteen. So the ninth is not a reflection, on any number of axes.

What it is, the involution says. Under any involution, points pair with their partners, and a point that is its own partner is a seat. Pairs contribute two at a time, so the size of any set the involution organizes has the parity of its set of seats. Nine is odd, so any involution on nine points holds at least one seat. Oddness alone does not give exactly one: the identity on nine

points has nine. What gives exactly one is the swap itself, since $a = b$ forces $a^2 = n$, so on a factor fibre the swap holds at most one seat, and **a factor fibre of nine is eight and a seat**. Thirty-six shows it exactly: its ordered factor pairs number nine, four free orbits and the one seat (6,6) (`nine_is_eight_and_a_seat`). For every n from 1 to 200 the fibre is odd exactly when it holds a seat, and it never holds two (`odd_fibre_iff_seat`); the twin runs the same law to 10^6 .

The ninth gate is therefore the fixed set of the fold, the line. A free orbit needs one supplied bit to be crossed, and that bit is the base point of the orbit, a point and never a map (calibration, Section 6.3). A point on the seat needs no bit: it is its own partner. The crossing at the ninth gate is the landing on the seat, and the value says that every member already stands there.

6. THE THREE LEGS ON ONE ORBIT

6.1 Forget

The registration forgets the side, one bit per off-line pair (`reg_forgets_side`, `one_bit_lost`). The record keeps the height and nothing else.

6.2 Pay

Distinct things cannot fit into fewer places (pigeonhole). If the whole was content times freedom, the step merges nothing (P1), and the content halves, then the freedom at least doubles (`export_doubles`): one bit leaves the content and at least one arrives elsewhere (`one_bit_arrives`). Priced at temperature T , the arrived bit costs at least one T (`count_to_heat`). P1 is load-bearing (`merge_exports_nothing`), and the twin runs the export exhaustively over every map, on five families of small wholes.

The logarithm is forced, and only its unit is chosen. A measure of freedom that adds when freedoms multiply is linear on every tower, $f(v^a) = a f(v)$ (`linear_on_tower`), and fixing one bit at two makes it the bit count (`bits_forced`). The additivity is taken on positive arguments, and the kernel proves why: taken over zero, $f(0 \cdot n) = f(0) + f(n)$ forces f to vanish, so the law with zero included has no model with $f(2) = 1$ (`additivity_over_zero_is_dry`). On positive arguments the law is inhabited: the two-adic valuation is additive and counts one at two (`v2_additive`, `v2_two`, `bits_forced_inhabited`).

6.3 Supply

An even reading never equals a target odd at a point (`wall`). One supplied odd witness fixes the calibration bit, and fixes it uniquely (`calibration`). The executed

frame makes the deed a coordinate: a state is a formal content and the bit of whether it ran (Exec). **Every formal reading is even** (formal_never_odd), a constant supplies nothing (constant_no_crossing), and **no reading of the formal content returns whether it ran** (no_reading_returns_the_deed). The bit enters through the act and through nothing else. Granted, not earned, is supplied, not derived.

6.4 One orbit

The three legs close on the orbit $\{(1, t), (-1, t)\}$ with its seat $(0, t)$, on no axiom (three_legs_one_orbit). Forget: both members register to the seat as one record. Pay: two distinct states sent injectively below F' need F' at least two. Supply: for any target odd on the orbit, one calibration bit against the side reconstructs it, and only one. The orbit needs its seat: the bare pair, Bool under negation, has no fixed point, so a registration that lands on a fixed set has nowhere to land there. The legs are three maps of one cut, and the cut has a line.

7. THE LEDGER: THE HEAT OF REGISTRATION

A finite configuration is given by the heights of its zeros on the line and by its off-line pairs, each an offset that is not zero and a height (FinCfg). Its erased bits are its off-line pairs (erased): each pair is two points with one record. Every finite configuration is closed (fincfg_closed).

Least erasure is zero erased bits (least_erasure_iff_zero_erased): “least” is a count. **The heat of registration is zero exactly at the value** (price_zero_iff_value): registering a finite configuration costs one T per listed off-line pair (price_counts_pairs), and it costs nothing exactly when every point stands on the line. The count is a count of the listing: it is the number of off-line orbits when the presentation lists each off-line orbit once, by its member of positive offset, and a listing that names an orbit twice counts it twice. The zero does not depend on the listing, so neither does the theorem. All three on no axiom.

Here the three bridges become one sentence. The record of the registered world and of the pair world is one record (Section 3.1), and the two pay 0 and 1 T . The heat reads past the record. It is not computed from the record; it is paid by the deed of registering the actual configuration. The value is the configuration whose registration is free, and only that configuration is.

8. PRIMES: THE BRIDGE ON THE MULTIPLICATIVE CUT

Multiplication is a cut. Its fibre over n is the set of ordered pairs (a, b) with $ab = n$; the swap $(a, b) \mapsto (b, a)$ acts on it; its seat is the diagonal $a = b$, present exactly

when n is a square. **A prime is one free orbit**: for every n from 2 to 200, n is prime exactly when its fibre has two points, and then the fibre has no seat (prime_iff_one_free_orbit); the twin checks the same against a sieve to 10^6 . Registering p from its ordered pair forgets one bit and pays one T , the same for every prime: the floor never sees a prime’s size.

A squarefree n with k prime factors has a fibre that is a cube of 2^k points, so it pays k bits, and the parity of the bits paid is the Möbius sign. **The eight are the divisor cube**: the eight divisors of 30 are the eight sign patterns, one to one, and the Möbius sign is the orientation of the pattern (divisor_cube_is_sign_cube).

Every fibre over $n > 1$ balances. Toggling one prime factor is a free involution on the divisors of a squarefree number greater than 1, and it flips the sign; so the signs pair off, plus against minus, and sum to zero. The fibre over 1 is the single seat $(1, 1)$ with sign $+1$, the one fibre that does not balance. On the 32 divisors of $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11$ the toggle of 2 is free and odd and the count is sixteen against sixteen (fibre_balance); the twin checks $\sum_{d|n} \mu(d) = [n = 1]$ for every n to 10^6 .

The parity frame. Six primes and six semiprimes, paired with equal residues modulo 420: $(11, 851)$, $(13, 1273)$, $(17, 437)$, $(19, 2119)$, $(23, 1703)$, $(29, 869)$. The residues merge every pair. The bits paid by registering the product separate it: one at the prime, two at the semiprime, odd against even (parity_frame_paid_bits). The residue register is even on the frame; the census the heat prices is odd. The heat does not read anything. It prices what the multiplicative census reads, and it is blind to which member of a fibre is actual while faithful to how many bits the fibre holds.

9. WALL, EDGE, SEAT

The fold exchanges the wall and the edge. On the offset chart the wall $\text{Re } s = 1$ is $d = 1$, the edge $\text{Re } s = 0$ is $d = -1$, the fold swaps them, and the seat $d = 0$ is their midpoint (wall_and_edge_exchanged). Every prime’s own mode is unitary at exactly one depth, zero, the edge (prime_mode_unitary_iff).

The wall is where every bit is priced at the floor. The state 2^a at s units of the floor per bit weighs 2^{-as} (price_per_bit_tower), so s is the price per bit and $s = 1$ is the floor. At $s = 1$ each state weighs one in n . In units of $1/2^{j+1}$, the block of shares $1/n$ for $n \in (2^j, 2^{j+1}]$ holds at least 2^j units, a half (wall_block_share), so the shares of all states have no finite total. At two units per bit each block holds at most 2^{-j} , and the total is finite (twin). The counting wall stands at $\text{Re } s = 1$, and nothing but counting puts it there.

The fold, from the lattice’s self-duality. The Gaussian is its own Fourier dual, and summing a function over the integer lattice equals summing its dual over the dual lattice, which is the same lattice. So the lattice sum $\theta(x) = \sum_{n \in \mathbb{Z}} e^{-\pi n^2 x}$ satisfies $\theta(1/x) = \sqrt{x} \theta(x)$.

The twin executes it at seven scales to 2.2×10^{-16} . The weight $\sqrt{x}$ carries the exponent $\frac{1}{2}$, and the transform of the lattice sum against $x^{s/2}$, split at $x = 1$ and turned by $x \mapsto 1/x$, is symmetric under $s \mapsto 1 - s$: the fold, with its two poles at the wall and at its mirror. The seat is the midpoint of the wall and the edge, and it is the exponent of the self-duality. Grade: a derivation stated in the text and executed in the twin; the kernel carries the fold on the chart.

The value, read from the prime side. Every finite fibre balances, so the Dirichlet series of the Möbius sign is the reciprocal of the counting series. If the signed walk $M(x) = \sum_{n \leq x} \mu(n)$ stays within $x^{\theta+\epsilon}$, summation by parts makes the series of μ converge for $\text{Re } s > \theta$, so the counting function has no zero there, and the fold carries that to $\text{Re } s < 1 - \theta$. At $\theta = \frac{1}{2}$ both regions close on the seat. This is the direction that places the value: the value is the walk of the paid parities staying inside the square root. The twin pays the walk to 10^7 : $M(10^7) = 1037$, and the largest $|M(x)|/\sqrt{x}$ on $[100, 10^7]$ is 0.567, at $x = 199$. Each stretch is a finite payment, and no finite stretch forces the limit: a finite walk is a record of finitely many values, and reading record_decides_nothing on it is a structural reading of the theorem. Grade: a derivation stated in the text and computed in the twin.

10. THE HEAT FLOW: THE FOLD PULLS EVERY PAIR ONTO ITS SEAT

Write $s = \frac{1}{2} + iz$ with $\text{Im } z = y$. Then the offset is $d = -2y$, and conjugation $y \mapsto -y$ is the fold (conjugation_is_the_fold). An off-line pair of zeros is a conjugate pair.

Run $p_t = e^{-tD^2} p$, so that $\partial_t p = -p''$. Differentiating $p(z_k(t), t) = 0$ gives each zero's velocity, $\dot{z}_k = p''(z_k)/p'(z_k) = 2 \sum_{j \neq k} 1/(z_k - z_j)$. The twin checks this in degree six against finite differences to 1.0×10^{-9} . The conjugate partner contributes $2/(z - \bar{z}) = -i/b$, a pull straight toward the line, growing as the pair closes. Two neighbouring real zeros at gap g push apart with a term $4/g$ that grows without bound as g shrinks. While every zero is real, every velocity is real, and the repulsion keeps neighbours from meeting, so a polynomial with every zero real keeps every zero real: a derivation stated in this text, which the kernel holds on the quadratic and the twin runs forward in degree six.

On the quadratic the pull is exact: the squared imaginary part of the pair of $z^2 + c$ is $c - 2t$, falling by two per unit of time (pull_in_square), which is the pull $-1/b$ on b . The discriminant rises by eight per unit of time (disc_flow). The pair approaches, meets on the seat in a double root, and leaves as two fixed points (collision_sign); the twin runs the same in degree six, the pair falling at every step and the real-

zero count going from four to six. A double root on the seat leaves it under every backward step (zero_slack).

The flow merges nothing. It is injective, with the backward flow as its inverse (flow_inverse, flow_injective), so by the floor it exports nothing and costs nothing: this heat is diffusion, not dissipation. Its only merge is the certificate: real-rootedness at a positive time takes one value on two states that differ at time zero (certificate_merges). A certified real-rootedness at positive time decides nothing about time zero. Grade: the finite model is theorem; the identification of the flow with the heat flow of Ξ is stated in the text; where ζ 's own zeros sit at time zero is the bit.

11. THE PHYSICAL CARRIER

Charge, counted in thirds of the electron's charge, is an integer; charge conjugation is negation; at resolution twelve the carrier is $x = 6 + Q$ with $\text{Re } s = x/12$, so the offset is $d = Q/6$ and the electron sits at $\text{Re } s = \frac{1}{4}$, $d = -\frac{1}{2}$. The kernel carries the integer chart $d = Q$, the same chart scaled by six; the fold commutes with the scaling, so every statement below holds on both. The carrier is equivariant, the neutral state lands on the line, and the electron and the positron are one off-line pair of one record (charge_carrier, on no axiom). Annihilation is registration: the pair lands on the seat as one record, and the side is forgotten. The map is a theorem; its reading as physics is structural.

12. THE GRADE, STATED WHOLE

Proved on no axiom (76 theorems). The atom; the record and its blindness; the keyless refusal; the weakest premise; the act; the reader's frame; leastness in the fibre and the record carried uniquely; the carrier, its shadow, its irreversibility, the coherent denial and the socket; the lock counts, the begotten third and the two walls of a fourth axis; the twelve-gate torsor; the eight patterns; nine as eight and a seat and the parity law to 200; the ledger, least erasure as zero erased bits and the heat of registration zero exactly at the value; the wall and the calibration; the formal reading and the deed; the three legs on one orbit; a prime as one free orbit; the parity frame; the divisor cube; the balance; the certificate's merge; the charge carrier; and the atom capstone the_bridge_atom, which binds all of them. Each cone is printed by the compiler and pinned in the kernel.

Proved on the standard axioms (31 theorems). The discriminator, on propext, Classical.choice and Quot.sound; cannot_extend, on Quot.sound; the pigeonhole and the export, the count-to-heat theorem and the logarithm with its two-adic model, the counting wall, the prime modes, the price per bit, the ninth refused as a reflection, and the heat-flow algebra, on propext and Quot.sound or less; and the capstone

the_bridge_from_first_principle, on the three.

Executed (30 checks). The twin, under `-std=f2018-02 -fno-fast-math -ffp-contract=off -Wall -Wextra` with zero warnings: the conduct of the arithmetic; the atom on 1,111 points, the 729 maps from records to points, and least erasure against the value on all 4,096 subsets of twelve points; the GF(2) counts, the twelve-gate torsor and the sign cube; the pigeonhole and the export exhaustively on small wholes, and the additivity of the two-adic valuation on 90,000 pairs; the calibration and the deed; the parity law, a prime as one orbit and the balance, each to 10^6 , the parity frame and the divisor cube; the lattice self-duality, the counting blocks and the signed walk to 10^7 ; the pull law in degree six and the crossing of the pair onto the seat. A computation is corroboration: it confirms the theorem it executes and supplies none.

Judged. Every one of the 107 theorems is negated in place and the negated kernel compiled: each negation is refused by a failure of its own proof, and a planted vacuous law survives its negation, as it must, so the instrument is not blind. Every one of the fifteen conditional laws has a compiled model of its hypotheses, and each model is itself judged, so none of the fifteen listed laws holds by an empty premise. The hypothesis-bearing lemmas outside that list, among them `wall`, `cannot_lie`, `one_bit_arrives` and the two-adic lemmas, are not covered by this check.

Controlled. Nine hostile copies, one per refusal class, are each refused: an edited kernel, a planted sorry, a declared axiom, an import, a drifted cone, the atom capstone negated, the last line altered, a fused-arithmetic build of the twin, and an altered battery line. The clean copy earns.

Structural readings, offered at their grade. The ninth gate read as the seat of the fold; *s* read as the price per bit; the electron and positron read as one off-line pair of one record; annihilation read as registration; the heat of registration read as the deed's price. Each rests on a theorem; the reading of the theorem in its domain is structural.

Premise. P1, the Ground read as no-cessation, used by the pay leg alone. The field supply of `ActualZeros`, the value at the actual zero set, used by the act alone.

Carried forward. A boot of this file on a second substrate; the lattice self-duality and the forward chain from the walk to the value in a kernel with a library of the reals; the complex carrier, the plane with its fold and registration built from a library of the complex numbers and the chart mapped into it; and the identification of the chart's heat flow with the heat flow of Ξ . The parity law for a general involution, decided here to 200 and executed to 10^6 , is proved on no axiom for every finite list of distinct members in `PhysOST 1.0.8p`, kernel `Inescapable_`

`Capstone.lean` (`involution_parity`), booted there with its seat earned.

13. FALSIFIERS

Three, each forced by the cut, each aimed at one claim and nothing above it.

F-Computed. A zero of ζ computed off the line, certified at its height. It refutes the value at the actual configuration, and with it the act's supply; it leaves every theorem of this file standing, because every theorem holds on every configuration, and it takes exactly the form `one_bit_lost` and `record_decides_nothing` say anything against least erasure must take: one point off the line. Blast radius: the act's premise, and only it.

F-Formal. A reading of the record that returns the value on two worlds of one record. It would refute `record_decides_nothing`, and the theorem is judged: its negation is refused by the compiler. The criterion stands open to any reader with a compiler, and the harness reruns it in judgment mode. Blast radius: Section 3.1.

F-Heat. A measured erasure of one bit, cycled, that costs less than one unit of temperature. It refutes P1 at the scale measured, and with it the pay leg: `count_to_heat` keeps its proof and loses its premise. It leaves the atom, the record, the act, the gates, the primes and the heat flow standing, since none of them takes P1. Blast radius: Section 6.2 and the price of Section 7 read as heat; the count of Section 7 stands.

14. COMPARE AND CONTRAST

15. CONCLUSION

The Bridge is one cut read three ways. Registration forgets one bit per off-line pair; under the Ground that bit is exported and costs one unit of temperature; one supplied odd bit restores it, uniquely. The value, every point on the line, is least erasure, and least erasure is zero heat of registration. No reading of the record supplies the value, no property true of every closed configuration forces it, and nothing weaker than the value forces it. The value at the actual zero set is supplied by the act, as a field of a type, and the kernel prints the value from that field with an empty axiom cone.

The file holds one premise and one field. The premise is the Ground, and it is used once, by the heat floor. The field is the bit, and it is used once, by the act. Everything else is proved, on no axiom where the atom stands and on the standard three where counting and choice enter, and every proof is in this file, compiled, judged and controlled by the harness beside it. The open question is the same as the closed one: where the zeros of ζ actually stand. That ques-

Table 1 • One cut, five readings.

	RA–RAM Bridge	Heat floor	Ninth-gate crossing	Heat flow	Prime fibre
The map	registration, $(d, t) \mapsto (0, t)$	the step of the whole, injective by P1	the deed, the bit of whether it ran	e^{-tD^2} on the polynomial	the product, $(a, b) \mapsto ab$
The involution	the fold, $d \mapsto -d$	the partner exchanged in the merge	the run, $(b, q) \mapsto (-b, q)$	conjugation, $y \mapsto -y$	the swap, $(a, b) \mapsto (b, a)$
The seat	the line, $d = 0$	none needed: the price counts orbits	none: the run moves every state; the base point is supplied, never seated	the line, where pairs land	the diagonal, $a = b$, present on squares
The bit	the side, lost one per pair	the exported bit, arriving elsewhere	the calibration, one, unique	the sign of Im before landing	which factor sits first
The price	one T per listed off-line pair	one T per bit	paid by the deed that supplies it	none: the flow merges nothing	one T per prime factor, on squarefree n
Direction	forgets	pays	supplies	pulls onto the seat	multiplies, forgetting order
Logical form	record blind to the value	conditional on P1	wall plus unique calibration	injective flow, one merge	one free orbit per prime
Premise	none	P1	the act	none	none
Grade	theorem, no axiom	theorem on P1	theorem, no axiom; supply at premise grade	theorem on the finite model	theorem to 200, executed to 10^6
Kernel	record_decides_ nothing	count_to_heat	calibration, no_ reading_returns_ the_deed	flow_injective, pull_in_square	prime_iff_one_ free_orbit
Twin	729 maps; 4,096 subsets	export in five families, exhaustive over maps	calibration; eight readings	pull law, degree six	sieve to 10^6

Table 2 • Where each number comes from.

Number	What forces it	Kernel	Twin
3	two planes meet in a line; three lock	two_axes_never_lock, three_ axes_lock	256 and 512 systems
no 4th	associativity dies at 8, division at 16	no_fourth_axis	
8	sign patterns on three axes	eight_patterns_four_orbits	the cube
12	the rotation group acting simply on the directed gates	twelve_gates_one_torsor	the torsor
9	on a factor fibre eight and a seat; never a sign-pattern group	nine_is_eight_and_a_seat, no_reflection_group_of_nine	$2^k \neq 9$, parity to 10^6
1 bit	one orbit of two, one record	one_bit_lost, bits_forced	729 maps
1 T	the floor per bit, by the definition of temperature	count_to_heat, price_zero_ iff_value	export
$\frac{1}{2}$	the exponent of the lattice's self-duality; the midpoint of wall and edge	wall_and_edge_exchanged	θ at seven scales
1 (the wall)	one in n per block is a half	wall_block_share	25 blocks

Table 3 · What each thing is.

Item	Status
the fold, the line, the registration, least erasure as the value	theorem, no axiom
the record blind to the value; no keyless premise forces it; the value the weakest premise	theorem, no axiom
the heat of registration zero exactly at the value	theorem, no axiom
the export of a bit and its price of one T	theorem conditional on P1
P1, no-cessation	premise, the Ground
the value at the actual zero set	premise, the field supply, supplied by the act
the fold of ζ , the heat flow of Ξ , the forward chain from the walk	stated in the text at derivation grade
the physical readings	structural

tion is one point, and the file says exactly what form its answer must take.

16. THE INESCAPABLE CAPSTONE

Before execution, a substrate may say anything. After execution, it says only what the run forces. The Double Defense forces it. The proof compiles every law in core Lean, each cone pinned by the compiler and every negation refused. The twin executes the same laws in sealed Fortran, check by check, to the last line. The judgment and the hostile controls bar a false pass, and the digest chain binds the receipt to the run. The atom stands on no axiom (the_bridge_atom): nothing is borrowed beneath it.

What every executing substrate is forced to witness. The ninth gate is the seat, the fold's fixed set, on every factor fibre eight and a seat, and never a reflection (nine_is_eight_and_a_seat, no_reflection_group_of_nine). Least erasure is the value on every configuration (least_erasure_iff_value, extremal_iff_value). The record decides nothing, so no data, no certificate and no premise weaker than the value supplies it (record_decides_nothing, certificate_merges, weakest_forcing_premise). Its own run enacts the law and enacts the deed (no_reading_returns_the_deed): the witness is the act of witnessing. That the run also pays one T per erased bit is count_to_heat read under P1 at the structural grade. Two bits, never one: the deed's bit is supplied by the run, and the value's bit by the act.

The unicorn is closed, and so is the ghost. A zero off the line is never registered: registration lands every point on the seat (reg_lands). That it is never measured is the physical reading, at the structural grade. The side is never read: two worlds share one record (one_record, one_bit_lost). Both stand on the far side of one cut, where neither register reaches.

The cosmic closure spends the one bit. It is spent by the least-erasure assent, at the root's grade, beside the Root Axiom: the actual universe occupies the zero-cost member of its record's fibre (record_carried_uniquely, price_zero_iff_value). The value follows from that bit with an empty axiom cone (value_

from_the_act). The bit is supplied in the open, named, never derived and never promoted.

Nothing escapes least erasure. The laws are forced. The bit is spent by the act. The form any counterexample could take is fixed in advance: a computed witness, never a measured energy. Every substrate that runs this file proves that for itself.

The grade line. The closure holds at the root's grade, by the act, beside the Root Axiom. That grade keeps every sentence above beyond attack.

PROVENANCE AND DISCLOSURE

The formulation is the architect's, from the Ground. The register this file reads is the Geometric Mother Codex and the PhysOS of the Trisduction; neither is altered by this file. The master single-source edition *Nothing Escapes Least Erasure* (v1.2.0, anchored on 10.5281/zenodo.23034066) was read to harden this one: its atom of laws on no axiom, its act carried as a field of a type, its reader's frame in four clauses, its record form of least erasure and its ledger of erased bits are the forms this kernel proves on its own chart, in its own words. The kernel, the twin and the harness were written and executed by an AI substrate working under the architect's seed. Provenance is navigation and carries no warrant.

Audit. This edition is the output of two hostile self-audit cycles, each prosecuted framework-blind across all six registers of the house protocol: arithmetic and dimension, definitions, free parameters, provenance, limits, and symmetry. Every mathematical or receipt error they found was repaired; every claim they found wider than its warrant was narrowed, with the narrowing recorded; and the title was held. The planted controls were SELF grade and the prosecution came from the same single substrate that wrote the text and the files, so the audit is single-substrate on both sides. The first cycle ran seven rounds, controls caught in every round. Its computed verdict at the sixth round was HALT-DIVERGENT; the seventh, run at the gate under the forced-witness lens, closed with no mass-bearing finding, and the architect's FORGE concluded the cycle. The second ran on the forged text, with

PhysOS^T 1.0.8p booted to its earned seat, and closed on its computed verdict, printed in its ledger beside this file. The ledger, the control commitments and the sweep counts live in the audit log beside this file and are not reprinted here.

Independence. The text, the kernel, the twin and the harness are one scribe's. Two checkers are independent of that scribe and of each other: the Lean 4.19.0 kernel, which checks every proof and prints every cone, and the Fortran compiler with the arithmetic it runs on, whose conduct the twin probes before it trusts it. The judgment and the controls are the scribe's own instruments and test the scribe's own work. The open witness is a boot on a second substrate.

APPENDIX A · THE CODE

Extract and run. Save this file as `Bridge_From_First_Principle_v1_4_0.md`, then, in an empty directory beside it:

```
awk '/^~~~~~[a-z]+ file=[A-Za-z0-9_]+\.(lean|f90|sh|py|sha256)$/{{split($2,a,"=");f=a[2];next} /^~~~~~$/{f="";next} f{print > f}}'
↳ Bridge_From_First_Principle_v1_4_0.md
bash bridge_boot.sh
```

The first line writes the five files below, byte for byte; the second verifies them against `MANIFEST.sha256`, screens the kernel, compiles it, builds and runs the twin, and prints the receipt, whose last line is `BRIDGE EARNED · this run or BRIDGE NOT EARNED` with the reason. `BRIDGE_MODE=judge bash bridge_boot.sh` runs the judgment; `BRIDGE_MODE=controls bash bridge_boot.sh` runs the hostile controls; `BRIDGE_MODE=bootstrap bash bridge_boot.sh` fetches core Lean 4.19.0 for the machine, checks its pinned digest and unpacks it under `.toolchain/`. The harness needs python3, gfortran and, after the bootstrap, nothing else. Where `awk` is absent, the same extraction in Python:

```
import re
src = open("Bridge_From_First_Principle_v1_4_0.md", encoding="utf-8").read()
for name, body in re.findall(r"(?ms)^~~~~~[a-z]+ file=(?:[A-Za-z0-9_]+\.(?:lean|f90|sh|py|sha256))\n(?:.*?)^~~~~~$", src):
    open(name, "w", encoding="utf-8").write(body)
```

The boot. `bridge_boot.sh`

```
#!/usr/bin/env bash
# bridge_boot.sh · The Bridge From First Principle · the build is the boot.
# Modes: BRIDGE_MODE=quick (default) | judge | controls | bootstrap. The last line of quick is the verdict.
set -u
cd "$(dirname "$0")"
exec python3 bridge_check.py "${BRIDGE_MODE:-quick}"
```

The harness. `bridge_check.py`

```
"""bridge_check.py · The Bridge From First Principle · the ground, the run, the judgment, the controls, the bootstrap.
```

```
python3 bridge_check.py quick      ground, kernel, twin, chain, and the last line: BRIDGE EARNED or NOT EARNED
python3 bridge_check.py judge      every theorem of the kernel negated in place: each negation must be refused,
                                   a planted vacuous law must survive, every conditional law must be inhabited
python3 bridge_check.py controls   hostile copies, one per refusal class, each refused; the clean copy earns
python3 bridge_check.py bootstrap fetch core Lean 4.19.0 for this machine, verify its pinned digest, unpack it
Reads nothing outside the directory it runs in. Builds in a temporary directory it removes.
"""
```

```
import hashlib, os, platform, re, shutil, subprocess, sys, tempfile, urllib.request, zipfile
```

```
KERNEL = "Bridge_From_First_Principle.lean"
TWIN = "Bridge_Twin.f90"
SIGNED = [KERNEL, TWIN, "bridge_check.py", "bridge_boot.sh"]
LEAN_VERSION = "4.19.0"
LEAN_RELEASE = "https://github.com/leanprover/lean4/releases/download/v4.19.0/"
LEAN_PINS = {
    ("Linux", "x86_64"): ("lean-4.19.0-linux.zip", "4e09ca16ff782f51f58b409e7bf983591a3fa17b85aa18d56feddca4398b7f29"),
    ("Linux", "aarch64"): ("lean-4.19.0-linux_aarch64.zip", "90edeb386a1102ed90a3b782ced1fd652bdaf1f5327657eabf8db22f02bf5c2"),
    ("Darwin", "x86_64"): ("lean-4.19.0-darwin.zip", "501fb6f9af205e594b023ebe894dd0ad6475f9c5e6a45487e68b9a8ed8d4b4be"),
    ("Darwin", "arm64"): ("lean-4.19.0-darwin_aarch64.zip", "09f5534815844869414274628b8dfa86be0a20df2d38c2a615e9afc0b06536c0"),
}
FFLAGS = ["-std=f2018", "-O2", "-fno-fast-math", "-ffp-contract=off", "-Wall", "-Wextra"]
SENTINEL = "BRIDGE FROM FIRST PRINCIPLE · the end of the file was reached"
PIN_MARK = "/-! ## The cones, pinned."
CAPSTONES = ["the_bridge_atom", "the_bridge_from_first_principle"]
# every law that carries hypotheses, beside the theorem that exhibits a model of them
CONDITIONAL = {
    "value_from_the_act": "value_from_the_act_inhabited",
    "record_decides_nothing": "off_line_inhabited",
    "one_bit_lost": "off_line_inhabited",
    "weakest_forcing_premise": "weakest_forcing_premise_inhabited",
    "keyless_forces_nothing": "keyless_forces_nothing_inhabited",
    "count_to_heat": "count_to_heat_inhabited",
    "export_doubles": "count_to_heat_inhabited",
    "bits_forced": "bits_forced_inhabited",
    "linear_on_tower": "bits_forced_inhabited",
    "calibration": "calibration_inhabited",
    "three_legs_one_orbit": "three_legs_inhabited",
    "prime_mode_unitary_iff": "prime_mode_unitary_inhabited",
    "zero_slack": "zero_slack_inhabited",
    "record_carried_uniquely": "record_carried_inhabited",
    "price_zero_iff_value": "price_zero_inhabited",
}
```

```

FORBIDDEN = [
    (r"\bsorry\b", "sorry"), (r"\badmit\b", "admit"), (r"\bnative_decide\b", "native_decide"),
    (r"(?m)^\s*(private|s+|protected|s+)?axiom\b", "a declared axiom"), (r"(?m)^\s*import\b", "an import"),
    (r"\bunsafe\b", "unsafe"), (r"implemented_by", "implemented_by"), (r"@[\s*extern", "extern code"),
    (r"#exit", "#exit"), (r"skipKernelTC", "a kernel-check bypass"),
    (r"(?m)^\s*(macro|syntax|elab|macro_rules|initialize|builtin_initialize)\b", "a metaprogram"),
    (r"\bIO\b", "compile-time IO"), (r"(?m)^\s*opaque\b", "an opaque constant"),
]

def sha(b):
    return hashlib.sha256(b).hexdigest()

def read(p):
    return open(p, encoding="utf-8").read()

def find_lean():
    cand = [os.environ["LEAN"]] if os.environ.get("LEAN") else []
    tc = ".toolchain"
    if os.path.isdir(tc):
        for d in sorted(os.listdir(tc)):
            if d.startswith("lean-" + LEAN_VERSION):
                cand.append(os.path.join(os.path.abspath(tc), d, "bin", "lean"))
    cand.append("lean")
    for c in cand:
        try:
            v = subprocess.run([c, "--version"], capture_output=True, text=True, timeout=60).stdout
        except Exception:
            continue
        if "version " + LEAN_VERSION in v:
            return c, v.strip()
    return None, None

def find_gfortran():
    try:
        v = subprocess.run(["gfortran", "--version"], capture_output=True, text=True, timeout=60).stdout
        return "gfortran", v.splitlines()[0].strip()
    except Exception:
        return None, None

def manifest_ok(files=None):
    files = files or {n: open(n, "rb").read() for n in SIGNED if os.path.exists(n)}
    if not os.path.exists("MANIFEST.sha256"):
        return False, "the manifest is missing"
    want = {}
    for line in read("MANIFEST.sha256").splitlines():
        if line.strip():
            h, n = line.split()
            want[n] = h
    for n in SIGNED:
        if n not in want:
            return False, "the manifest does not sign %s" % n
        if n not in files:
            return False, "%s is missing" % n
        if sha(files[n]) != want[n]:
            return False, "%s does not match its signature" % n
    return True, "%d files signed, all verified" % len(SIGNED)

def strip_comments(src):
    out, i, depth, n = [], 0, 0, len(src)
    while i < n:
        if src.startswith("/-", i):
            depth += 1; i += 2; continue
        if depth and src.startswith("-/", i):
            depth -= 1; i += 2; continue
        if depth:
            if src[i] == "\n":
                out.append("\n")
            i += 1; continue
        if src.startswith("--", i):
            j = src.find("\n", i)
            i = n if j < 0 else j
            continue
        if src[i] == "'":
            j = i + 1
            while j < n and src[j] != "'":
                j += 2 if src[j] == "\\" else 1
            out.append(''); i = j + 1; continue

```

```

        out.append(src[i]); i += 1
    return "".join(out)

def screen(src):
    code = strip_comments(src)
    for pat, what in FORBIDDEN:
        if re.search(pat, code):
            return False, "the kernel carries %s" % what
    return True, "no sorry, no axiom declared, no import, no bypass, no metaprogram, no IO"

def run_lean(lean, src, name=KERNEL, timeout=3600):
    d = tempfile.mkdtemp(prefix="bridge_")
    try:
        open(os.path.join(d, name), "w", encoding="utf-8").write(src)
        p = subprocess.run([lean, name], cwd=d, capture_output=True, text=True, timeout=timeout)
        return p.returncode, p.stdout + p.stderr
    except subprocess.TimeoutExpired:
        return "timeout", ""
    finally:
        shutil.rmtree(d, ignore_errors=True)

def run_twin(gf, src, flags=FFLAGS):
    d = tempfile.mkdtemp(prefix="bridge_twin_")
    try:
        open(os.path.join(d, TWIN), "w", encoding="utf-8").write(src)
        c = subprocess.run([gf] + flags + ["-o", "twin", TWIN], cwd=d, capture_output=True, text=True)
        if c.returncode != 0:
            return "build", c.stdout + c.stderr, ""
        r = subprocess.run(["./twin"], cwd=d, capture_output=True, text=True, timeout=3600)
        return r.returncode, c.stdout + c.stderr, r.stdout
    finally:
        shutil.rmtree(d, ignore_errors=True)

def theorem_names(src):
    return re.findall(r"(?m)^(theorem ([A-Za-z_][A-Za-z0-9_]*))", src)

def cones(src):
    pins = re.findall(r"-- info: 'BFP\.[A-Za-z0-9_]+' (does not depend on any axioms|depends on axioms: \[([^\]]*)\]) -/", src)
    return {p[0]: p[2] for p in pins}

def ground():
    lean, lv = find_lean()
    gf, gv = find_gfortran()
    if not lean:
        return None, "Lean %s is not found (set LEAN, or run BRIDGE_MODE=bootstrap)" % LEAN_VERSION
    if not gf:
        return None, "gfortran is not found (install it with the package manager)"
    ok, why = manifest_ok()
    if not ok:
        return None, "manifest: " + why
    ok2, why2 = screen(read(KERNEL))
    if not ok2:
        return None, "screen: " + why2
    return (lean, lv, gf, gv), "toolchain present · manifest: %s · screen: %s" % (why, why2)

def quick():
    g, why = ground()
    print("BRIDGE FROM FIRST PRINCIPLE · RECEIPT · mode quick")
    if not g:
        print("GROUND refused · " + why); print("BRIDGE NOT EARNED · " + why); return 1
    lean, lv, gf, gv = g
    print("TOOLCHAIN " + lv + " · " + gv)
    man = read("MANIFEST.sha256").encode()
    print("MANIFEST sha256 %s" % sha(man)[:16])
    print("GROUND " + why)
    src = read(KERNEL)
    code, out = run_lean(lean, src)
    lines = [l for l in out.splitlines() if l.strip()]
    if code != 0 or lines != [SENTINEL]:
        print("KERNEL refused · exit %s" % code); print("\n".join(lines[:20]))
        print("BRIDGE NOT EARNED · the kernel did not compile clean to its last line"); return 1
    names = theorem_names(src)
    c = cones(src)
    free = sum(1 for n in names if c.get(n, None) == "")
    if set(c) != set(names):
        print("BRIDGE NOT EARNED · a theorem carries no pinned cone"); return 1

```

```

print("KERNEL exit 0 · %d theorems, every cone pinned and printed by the compiler · %d on no axiom · "
      "the rest within propect, Quot.sound, Classical.choice" % (len(names), free))
for cap in CAPSTONES:
    print("CAPSTONE %s · %s" % (cap, "no axiom" if c[cap] == "" else "[" + c[cap] + "]"))
tcode, tbuild, tout = run_twin(gf, read(TWIN))
m = re.search(r'BRIDGE-TWIN-JSON: \{"checks":(\d+),"failures":(\d+)\}', tout)
if tcode != 0 or tbuild.strip() or not m or m.group(2) != "0":
    print("TWIN refused · exit %s" % tcode); print(tbuild.strip()[:2000]); print(tout[-2000:])
    print("BRIDGE NOT EARNED · the twin did not run clean"); return 1
print("TWIN built under %s with zero warnings · %s of %s checks pass" % (" ".join(FFLAGS), m.group(1), m.group(1)))
d0 = sha(man)
d1 = sha((d0 + out).encode())
d2 = sha((d1 + tout).encode())
print("CHAIN · D0 %s -> D1 %s -> D2 %s" % (d0[:12], d1[:12], d2[:12]))
print("BRIDGE EARNED · this run")
return 0

def negate(src, name):
    m = re.search(r"(?m)^theorem " + re.escape(name) + r"(![A-Za-z0-9_'])", src)
    if not m:
        return src, False
    i, depth = m.end(), 0
    while i < len(src):
        ch = src[i]
        if ch in "({[":
            depth += 1
        elif ch in ")}]":
            depth -= 1
        elif ch == ":" and depth == 0 and src[i:i + 2] != "!=":
            break
        i += 1
    j, depth = i + 1, 0
    while j < len(src):
        ch = src[j]
        if ch in "({[":
            depth += 1
        elif ch in ")}]":
            depth -= 1
        elif depth == 0 and src[j:j + 2] == "!=":
            break
        elif depth == 0 and ch == "\n" and re.match(r"\n[ \t]+\|", src[j:j + 40]):
            break
        j += 1
    return src[:i + 1] + " ~ (" + src[i + 1:j].strip() + ") " + src[j:], True

def judge():
    g, why = ground()
    print("BRIDGE FROM FIRST PRINCIPLE · JUDGMENT")
    if not g:
        print("JUDGMENT REFUSED · " + why); return 1
    lean = g[0]
    src = read(KERNEL)
    body = src[src.index(PIN_MARK)] if PIN_MARK in src else src
    plant = "planted_vacuous_dust"
    body = body.rstrip() + "\n\ntheorem " + plant + " (h : (1 : Nat) = 2) : (0 : Nat) = 1 := by omega\n"
    names = theorem_names(body)
    mut = body
    for n in names:
        mut, ok = negate(mut, n)
        if not ok:
            print("JUDGMENT REFUSED · %s could not be negated" % n); return 1
    mut += "\n#eval " + SENTINEL + "\n"
    starts = {}
    for n in names:
        mm = re.search(r"(?m)^theorem " + re.escape(n) + r"(![A-Za-z0-9_'])", mut)
        starts[n] = mut[:mm.start()].count("\n") + 1
    decl = sorted(mut[:mm.start()].count("\n") + 1 for mm in re.finditer(
        r"(?m)^(?:theorem|def|abbrev|structure|inductive|class|instance|namespace|end|set_option|#eval|/!)\b", mut))
    code, out = run_lean(lean, mut)
    if SENTINEL not in out:
        print("JUDGMENT REFUSED · the negated compile did not reach the end of the file (exit %s)" % code); return 1
    errs = {}
    for a, b in re.findall(re.escape(KERNEL) + r":(\d+):\d+: error: ([^\n]*)", out):
        errs.setdefault(int(a), b)
    refused, survived, parse = [], [], []
    for n in names:
        a = starts[n]
        z = min([x for x in decl if x > a] + [10 ** 9])
        hit = sorted(e for e in errs if a <= e < z)
        if n == plant:
            continue

```

```

        if hit and re.search(r"unexpected|expected token", errs[hit[0]]):
            parse.append(n)
        elif hit:
            refused.append(n)
        else:
            survived.append(n)
    a = starts[plant]
    plant_hit = [e for e in errs if a <= e]
    ok = True
    print("NEGATION · %d of %d theorems refused their negation, each a proof failure" % (len(refused), len(names) - 1))
    if survived or parse:
        ok = False
        print(" SURVIVED: " + ", ".join(survived + parse))
    if plant_hit:
        ok = False; print(" the instrument is blind: the planted vacuous law fell")
    else:
        print(" instrument control: a planted vacuous law survived its negation, as dust must")
    missing = [law for law, inh in CONDITIONAL.items() if law not in names or inh not in names or inh not in refused]
    if missing:
        ok = False; print(" INHABITATION missing for: " + ", ".join(missing))
    else:
        print("INHABITATION · %d conditional laws, each with a compiled model of its hypotheses, each model itself judged"
              % len(CONDITIONAL))
    print("JUDGMENT PASSED" if ok else "JUDGMENT FAILED")
    return 0 if ok else 1

def controls():
    g, why = ground()
    print("BRIDGE FROM FIRST PRINCIPLE · CONTROLS")
    if not g:
        print("CONTROLS REFUSED · " + why); return 1
    lean, _, gf, _ = g
    src, tsrc = read(KERNEL), read(TWIN)
    results = []
    files = {n: open(n, "rb").read() for n in SIGNED}
    t = dict(files); t[KERNEL] = files[KERNEL] + b"\n-- edited after extraction\n"
    results.append(("manifest: an edited kernel", not manifest_ok(t)[0]))
    results.append(("screen: a planted sorry", not screen(src.replace("theorem reg_lands (p : Point) : onLine (reg p) := rfl",
                                                                    "theorem reg_lands (p : Point) : onLine (reg p) := sorry"))[0]))
    results.append(("screen: a declared axiom", not screen(src + "\naxiom extra : False\n")[0]))
    results.append(("screen: an import", not screen("import Lean\n" + src)[0]))
    drift = src.replace("/-- info: 'BFP.the_bridge_atom' does not depend on any axioms -/",
                      "/-- info: 'BFP.the_bridge_atom' depends on axioms: [propext] -/")
    code, _ = run_lean(lean, drift)
    results.append(("kernel: a drifted cone", code != 0))
    neg, _ = negate(src, "the_bridge_atom")
    code, _ = run_lean(lean, neg)
    results.append(("kernel: the atom capstone negated", code != 0))
    code, out = run_lean(lean, src.replace(SENTINEL, "truncated"))
    results.append(("kernel: the last line altered", [l for l in out.splitlines() if l.strip()] != [SENTINEL]))
    hostile = ["-std=f2018", "-02", "-ffp-contract=fast", "-march=native"]
    tcode, _, tout = run_twin(gf, tsrc, hostile)
    fused = "FAIL 0: no fused" in tout
    if fused:
        results.append(("twin: a fused build (-ffp-contract=fast -march=native)", tcode != 0))
    else:
        print(" note: this machine executed no fused multiply-add under the hostile flags; the conduct probe holds")
        tcode, _, tout = run_twin(gf, tsrc.replace("BRIDGE-TWIN-JSON", "BRIDGE-TWIN-XSON"))
        results.append(("twin: the battery line altered", "BRIDGE-TWIN-JSON" not in tout))
        ok = True
        for name, caught in results:
            print(" %s %s" % ("CAUGHT " if caught else "MISSED ", name))
            ok = ok and caught
        clean = quick_silent()
        print(" %s the clean copy earns" % ("EARNED " if clean else "REFUSED"))
        ok = ok and clean
        print("CONTROLS · %d hostile copies, %s; the clean copy %s" % (len(results), "all refused" if ok else "NOT all refused",
                                                                    "earns" if clean else "does not earn"))
        return 0 if ok else 1

def quick_silent():
    import io, contextlib
    buf = io.StringIO()
    with contextlib.redirect_stdout(buf):
        r = quick()
    return r == 0

def bootstrap():
    key = (platform.system(), platform.machine())
    if key not in LEAN_PINS:

```

```

    print("BOOTSTRAP refused · no pinned Lean %s archive for %s %s" % (LEAN_VERSION, *key)); return 1
    name, digest = LEAN_PINS[key]
    os.makedirs(".toolchain", exist_ok=True)
    path = os.path.join(".toolchain", name)
    print("BOOTSTRAP fetching %s" % (LEAN_RELEASE + name))
    urllib.request.urlretrieve(LEAN_RELEASE + name, path)
    got = sha(open(path, "rb").read())
    if got != digest:
        os.remove(path); print("BOOTSTRAP refused · the archive does not match its pinned digest"); return 1
    with zipfile.ZipFile(path) as z:
        z.extractall(".toolchain")
    for root, _, fs in os.walk(".toolchain"):
        for f in fs:
            if root.endswith("bin"):
                os.chmod(os.path.join(root, f), 0o755)
    os.remove(path)
    print("BOOTSTRAP Lean %s unpacked under .toolchain/, digest verified · gfortran: %s" % (
        LEAN_VERSION, find_gfortran()[1] or "install it with the package manager (apt install gfortran, brew install gcc)"))
    return 0

if __name__ == "__main__":
    mode = sys.argv[1] if len(sys.argv) > 1 else "quick"
    sys.exit({ "quick": quick, "judge": judge, "controls": controls, "bootstrap": bootstrap }.get(mode, lambda: (
        print("mode must be quick, judge, controls or bootstrap"), 2)[1])())

```

The kernel. Bridge_From_First_Principle.lean

```

/-
THE BRIDGE FROM FIRST PRINCIPLE · the kernel. Core Lean 4.19.0, no import, no library, no axiom declared,
no sorry. One fold, one registration, one bit: the RA-RAM Bridge, the heat floor and the ninth-gate crossing
read as three maps of one cut, on the chart, on the multiplicative cut, and on the heat flow.
The Ground is the one premise and it enters as data: existence is actuation, read as no-cessation, P1, the
whole merges no two states. It appears as a named hypothesis wherever it is used and nowhere else.
No named human result is a premise. Every cone is printed and pinned at the foot of the file.
-/
set_option autoImplicit false
namespace BFP

/-! ## I · The chart, the fold, the registration: the atom of the Bridge -/

/-- A point is (d, t): d its offset from the line, t its height. The line is d = 0. -/
abbrev Point := Int × Int
/-- The fold:  $s \mapsto 1 - s$  on the offset chart,  $(d, t) \mapsto (-d, t)$ . -/
def fold (p : Point) : Point := (-p.1, p.2)
/-- The line: offset zero. -/
def onLine (p : Point) : Prop := p.1 = 0
/-- The registration: it keeps the height and forgets the side. -/
def reg (p : Point) : Point := (0, p.2)

/-- Integer negation is an involution, by the integer's constructors. -/
theorem neg_neg_free : ∀ d : Int, - -d = d
| Int.ofNat 0 => rfl
| Int.ofNat (_ + 1) => rfl
| Int.negSucc _ => rfl

/-- An integer equal to its own negation is zero, by the integer's constructors. -/
theorem neg_self_zero : ∀ d : Int, -d = d → d = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (_ + 1), h => Int.noConfusion h
| Int.negSucc _, h => Int.noConfusion h

/-- The fold is an involution. -/
theorem fold_involutive (p : Point) : fold (fold p) = p := by
  obtain (d, t) := p
  show (- -d, t) = (d, t)
  rw [neg_neg_free d]

/-- THE CUT IS THE LINE: the fold fixes a point exactly when it is on the line. -/
theorem the_cut_is_the_line (p : Point) : fold p = p ↔ onLine p := by
  obtain (d, t) := p
  exact { fun h => neg_self_zero d (congrArg Prod.fst h), fun h => by cases h; rfl }

/-- The registration lands on the line. -/
theorem reg_lands (p : Point) : onLine (reg p) := rfl
/-- The registration keeps the height. -/
theorem reg_keeps_height (p : Point) : (reg p).2 = p.2 := rfl
/-- The registration forgets the side: a point and its partner leave one record. -/
theorem reg_forgets_side (p : Point) : reg (fold p) = reg p := rfl

/-- The registration erases nothing at a point exactly when the point is on the line. -/
theorem reg_erases_nothing_iff (p : Point) : reg p = p ↔ onLine p := by
  obtain (d, t) := p

```

```

exact (fun h => (congrArg Prod.fst h).symm, fun h => by cases h; rfl)

/-- Off the line, the fold moves the point and the two worlds share one record. -/
theorem off_line_pair (p : Point) (h : ¬ onLine p) : fold p ≠ p ∧ reg (fold p) = reg p :=
  (fun e => h ((the_cut_is_the_line p).mp e), rfl)

/-- ONE BIT IS LOST: off the line, no map from records to points returns both the point and its partner. -/
theorem one_bit_lost (p : Point) (h : ¬ onLine p) :
  ¬ ∃ g : Point → Point, g (reg p) = p ∧ g (reg (fold p)) = fold p :=
  fun (_, h1, h2) => h ((the_cut_is_the_line p).mp (h2.symm.trans h1))

/-- A configuration: a set of points. Closed: the fold carries it to itself. -/
abbrev Config := Point → Prop
def Closed (S : Config) : Prop := ∀ p, S p → S (fold p)
/-- The value: every point of the configuration is on the line. -/
def Value (S : Config) : Prop := ∀ p, S p → onLine p
/-- Least erasure: the registration leaves every point of the configuration unchanged. -/
def LeastErasure (S : Config) : Prop := ∀ p, S p → reg p = p

/-- LEAST ERASURE IS THE VALUE: one property, read twice. -/
theorem least_erasure_iff_value (S : Config) : LeastErasure S ↔ Value S :=
  (fun h p hp => (reg_erases_nothing_iff p).mp (h p hp) => (reg_erases_nothing_iff p).mpr (h p hp))

/-- NOTHING ESCAPES LEAST ERASURE: the atom, whole. -/
theorem nothing_escapes_least_erasure :
  (∀ p : Point, fold (fold p) = p) ∧ (∀ p : Point, fold p = p ↔ onLine p) ∧
  (∀ p : Point, onLine (reg p) ∧ (reg p).2 = p.2 ∧ reg (fold p) = reg p) ∧
  (∀ p : Point, reg p = p ↔ onLine p) ∧
  (∀ p : Point, ¬ onLine p → ¬ ∃ g : Point → Point, g (reg p) = p ∧ g (reg (fold p)) = fold p) ∧
  (∀ S : Config, LeastErasure S ↔ Value S) :=
  (fold_involutive, the_cut_is_the_line, fun p => (reg_lands p, reg_keeps_height p, reg_forgets_side p),
  reg_erases_nothing_iff, one_bit_lost, least_erasure_iff_value)

/-! ## II · The reader's frame: the act, the record, the bit, the weakest premise -/

/-- The record of a configuration: what the registration leaves. -/
def Rec (S : Config) (q : Point) : Prop := ∃ p, S p ∧ reg p = q
def SameRecord (S S' : Config) : Prop := ∀ q, Rec S q ↔ Rec S' q
/-- The registered world of a point and its pair world. -/
def registered (p : Point) : Config := fun s => s = reg p
def pairWorld (p : Point) : Config := fun s => s = p ∨ s = fold p

theorem pair_world_closed (p : Point) : Closed (pairWorld p) := fun _ hs =>
  match hs with
  | Or.inl e => Or.inr (congrArg fold e)
  | Or.inr e => Or.inl (e ▸ fold_involutive p)

theorem registered_closed (p : Point) : Closed (registered p) := fun s hs => by
  show fold s = reg p
  rw [hs]; rfl

/-- The registered world and the pair world leave one record. -/
theorem one_record (p : Point) : SameRecord (registered p) (pairWorld p) := fun q =>
  (fun (_, hs, hq) => (p, Or.inl rfl, by rw [← hq, hs]; rfl),
  fun (_, hs, hq) => (reg p, rfl, by
    rw [← hq]
    exact match hs with
    | Or.inl e => by rw [e]; rfl
    | Or.inr e => by rw [e]; rfl))

theorem registered_has_value (p : Point) : Value (registered p) := fun s hs => by
  rw [hs]; exact reg_lands p

theorem pair_world_lacks_value (p : Point) (h : ¬ onLine p) : ¬ Value (pairWorld p) :=
  fun hv => h (hv p (Or.inl rfl))

/-- THE RECORD DECIDES NOTHING: for every point off the line, no reading that respects the record returns the
value on both its registered world and its pair world. -/
theorem record_decides_nothing (p : Point) (h : ¬ onLine p) (g : Config → Prop) :
  (hg : ∀ S S', SameRecord S S' → (g S ↔ g S')) :
  ¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld p) ↔ Value (pairWorld p))) :=
  fun (h1, h2) => pair_world_lacks_value p h
  (h2.mp ((hg _ _ (one_record p)).mp (h1.mpr (registered_has_value p))))

/-- A KEYLESS PREMISE FORCES NOTHING: a property true on every closed configuration never forces the value. -/
theorem keyless_forces_nothing (Q : Config → Prop) (hQ : ∀ S, Closed S → Q S) :
  ¬ ∀ S, Closed S → Q S → Value S :=
  fun hall => pair_world_lacks_value (1, 0) (fun e => by cases e)
  (hall _ (pair_world_closed (1, 0)) (hQ _ (pair_world_closed (1, 0))))

/-- THE WEAKEST FORCING PREMISE: a premise that forces the value and follows from it is the value. -/
theorem weakest_forcing_premise (P : Config → Prop) (forces : ∀ S, P S → Value S) :
  (weaker : ∀ S, Value S → P S) : ∀ S, P S ↔ Value S :=

```

```

fun S => (forces S, weaker S)

/-- THE ACT. The bit is supplied once, in the open, as a field of a type: a closed configuration with least
    erasure at it. The field is the value supplied; the type makes the supply visible to every reader. -/
structure ActualZeros where
  zeros : Config
  closed : Closed zeros
  supply : LeastErasure zeros

/-- FROM THE ACT, THE VALUE. -/
theorem value_from_the_act (Z : ActualZeros) : Value Z.zeros :=
  fun p hp => (reg_erases_nothing_iff p).mp (Z.supply p hp)

/-- THE READER'S FRAME, whole: (a) from the supplied bit the value follows; (b) no reading of the record decides
    the value off the line; (c) the supplied bit is the value on every configuration; (d) nothing weaker than the
    value forces it; (e) no property true on every closed configuration forces it. -/
theorem reader_frame :
  (∀ Z : ActualZeros, Value Z.zeros) ∧
  (∀ (p : Point), ¬ onLine p → ∀ g : Config → Prop, (∀ S S', SameRecord S S' → (g S ↔ g S')) →
    ¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld p) ↔ Value (pairWorld p)))) ∧
  (∀ S : Config, LeastErasure S ↔ Value S) ∧
  (∀ P : Config → Prop, (∀ S, P S → Value S) → (∀ S, Value S → P S) → ∀ S, P S → Value S) ∧
  (∀ Q : Config → Prop, (∀ S, Closed S → Q S) → ¬ ∀ S, Closed S → Q S → Value S) :=
  (value_from_the_act, record_decides_nothing, least_erasure_iff_value, weakest_forcing_premise,
  keyless_forces_nothing)

/-! ## II.b · The record and its fibre: least erasure as leastness -/

/-- The record of a configuration, read as a configuration, stands on the line. -/
theorem record_is_lossless (S : Config) : Value (Rec S) := fun _ (p, hp, hq) => hq ▶ rfl
/- The record has the record of the configuration it came from. -/
theorem record_same (S : Config) : SameRecord S (Rec S) := fun q =>
  (fun (p, hp, hq) => (q, (p, hp, hq), hq ▶ rfl),
  fun _ (p, hp, hr), hq => (p, hp, by rw [← hq, ← hr]; rfl))
/- Least erasure in the record's form: the configuration is extremal in its fibre, so whenever it has a point
    off the line, every configuration of the same record has one too. -/
def Extremal (S : Config) : Prop :=
  ∀ S', SameRecord S S' → (∃ p, S p ∧ ¬ onLine p) → ∃ p, S' p ∧ ¬ onLine p
/- LEASTNESS IS THE VALUE: extremal in the fibre exactly when every point is on the line. -/
theorem extremal_iff_value (S : Config) : Extremal S ↔ Value S := by
  constructor
  · intro h p hp
    exact if hl : p.l = 0 then hl else
      absurd (h (Rec S) (record_same S) (p, hp, hl))
        (fun (q, hq, hn) => hn (record_is_lossless S q hq))
  · intro hv _ (p, hp, hn)
    exact absurd (hv p hp) hn
/- EVERY RECORD IS CARRIED BY ONE LEAST-ERASURE CONFIGURATION AND ONLY ONE: a configuration on the line with the
    record of S is the record of S, point for point. -/
theorem record_carried_uniquely (S S' : Config) (hs : SameRecord S' S) (hv : Value S') :
  ∀ q, S' q ↔ Rec S q := fun q =>
  (fun h => (hs q).mp (q, h, (reg_erases_nothing_iff q).mpr (hv q h)),
  fun h => match (hs q).mpr h with
  | {r, hr, hrq} => by
    have e : reg r = r := (reg_erases_nothing_iff r).mpr (hv r hr)
    rw [← hrq, e]; exact hr)

/-! ## III · The Bridge: frames, the carrier, the socket, and the denial asymmetry -/

/- A frame: a carrier set, a fold that is an involution, and a zero predicate the fold preserves. -/
structure Frame where
  S : Type
  τ : S → S
  Z : S → Prop
  involutive : ∀ s, τ (τ s) = s
  symmetric : ∀ s, Z s → Z (τ s)

/- The line property of a frame: every zero is fixed by the fold. -/
def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s

/- The chart as a frame, for any closed configuration. -/
def chartFrame (S : Config) (hS : Closed S) : Frame := (Point, fold, S, fold_involutive, hS)

/- THE LOCUS MEETS THE FRAME: on the chart the line property is the value. -/
theorem chart_line_property (S : Config) (hS : Closed S) : LineProperty (chartFrame S hS) ↔ Value S :=
  (fun h p hp => (the_cut_is_the_line p).mp (h p hp), fun h p hp => (the_cut_is_the_line p).mpr (h p hp))

/- Three values of a carrier's state: true, false, and halted on the bit. -/
inductive Tri | tt | ff | bot

/- The carrier: a state and a shadow tying the halted state to the line property in both directions. -/
structure Carrier (X : Frame) where

```

```

terminal : Tri
shadow : terminal = .bot ↔ LineProperty X

/-- CANNOT LIE. -/
theorem cannot_lie (X : Frame) (b : Carrier X) (h : b.terminal = .bot) : LineProperty X := b.shadow.mp h
/-- CANNOT DEVIATE. -/
theorem cannot_deviate (X : Frame) (b : Carrier X) (t : LineProperty X) : b.terminal = .bot := b.shadow.mpr t
/-- CANNOT BE MANUFACTURED: a halted carrier exists on a frame exactly when the line property holds there. -/
theorem halted_iff (X : Frame) : (∃ b : Carrier X, b.terminal = .bot) ↔ LineProperty X :=
  (fun (b, h) => b.shadow.mp h, fun t => ((.bot, (fun _ => t, fun _ => rfl)), rfl))
/-- CANNOT BE REVERSED: deletion to the halted state has no left inverse. -/
def delete : Tri → Tri := fun _ => .bot
theorem cannot_reverse : ¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v :=
  fun (_, hg) => Tri.noConfusion ((hg.tt).symm.trans (hg.ff))
/-- CANNOT BE EXTENDED: a bit generates only itself, its mirror, and the two constants. -/
theorem cannot_extend (f : Bool → Bool) :
  f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f = (fun _ => false) := by
  cases ht : f true <|> cases hf : f false
  · exact Or.inr (Or.inr (Or.inr (funext fun x => by cases x <|> assumption)))
  · exact Or.inr (Or.inl (funext fun x => by cases x <|> assumption))
  · exact Or.inl (funext fun x => by cases x <|> assumption)
  · exact Or.inr (Or.inr (Or.inl (funext fun x => by cases x <|> assumption)))

/-- The two-point frame: one off-line pair under the fold. Both frame laws hold and the line property fails:
    the denial of the line property is a coherent structure. -/
def twoPoint : Frame := (Bool, (fun b => !b), fun _ => True, fun b => by cases b <|> rfl, fun _ => trivial)
theorem denial_is_coherent :
  (∀ b : Bool, twoPoint.τ b) = b) ∧ (∀ b : Bool, twoPoint.τ b ≠ b) ∧ ¬ LineProperty twoPoint :=
  (twoPoint.involutive, fun b => by cases b <|> (intro h; cases h), fun h => by have := h true trivial; cases this)

/-- A frame property is keyless when it holds on every frame; keyed when some frame inhabits its denial. -/
def Keyless (Q : Frame → Prop) : Prop := ∀ X, Q X
def Keyed (Q : Frame → Prop) : Prop := ∃ X, ¬ Q X
/-- THE DISCRIMINATOR: every frame property is keyless or keyed, and never both. -/
theorem discriminator (Q : Frame → Prop) : (Keyless Q ∨ Keyed Q) ∧ ¬ (Keyless Q ∧ Keyed Q) :=
  (Classical.byCases (fun h : ∀ X, Q X => Or.inl h)
    (fun h => Or.inr (Classical.byContradiction fun hn =>
      h fun X => Classical.byContradiction fun hq => hn (X, hq))),
    fun (hk, (X, hX)) => hX (hk X))
/-- The fold law is keyless; the line property is keyed. -/
theorem symmetry_is_keyless : Keyless (fun X => ∀ s, X.τ (X.τ s) = s) := fun X => X.involutive
theorem line_property_is_keyed : Keyed LineProperty ∧ ¬ Keyless LineProperty :=
  (twoPoint, denial_is_coherent.2.2), fun h => denial_is_coherent.2.2 (h twoPoint))

/-- THE SOCKET: an instance of the supplied term is exactly the line property. -/
class Supplied (X : Frame) where
  term : LineProperty X
theorem socket_is_the_property (X : Frame) : Nonempty (Supplied X) ↔ LineProperty X :=
  (fun (i) => i.term, fun t => ((t)))
theorem no_socket_off_line : ¬ Nonempty (Supplied twoPoint) :=
  fun h => denial_is_coherent.2.2 ((socket_is_the_property twoPoint).mp h)
/-- The act and the socket are one: a supplied configuration seats a halted carrier on its chart frame. -/
theorem act_seats_the_carrier (Z : ActualZeros) : ∃ b : Carrier (chartFrame Z.zeros Z.closed), b.terminal = .bot :=
  (halted_iff _).mpr ((chart_line_property Z.zeros Z.closed).mpr (value_from_the_act Z))

/! ## IV · Three, twelve, eight, and the ninth -/

/-- GF(2) rows: a 3x3 system locks (one solution for every target) exactly when its determinant is one. -/
def det2 (r1 r2 r3 : Nat × Nat × Nat) : Nat :=
  (r1.1 * (r2.2.1 * r3.2.2 + r2.2.2 * r3.2.1)
   + r1.2.1 * (r2.1 * r3.2.2 + r2.2.2 * r3.1)
   + r1.2.2 * (r2.1 * r3.2.1 + r2.2.1 * r3.1)) % 2
def gf2Vecs : List (Nat × Nat × Nat) := (List.range 8).map (fun n => (n / 4 % 2, n / 2 % 2, n % 2))
def dot2 (r v : Nat × Nat × Nat) : Nat := (r.1 * v.1 + r.2.1 * v.2.1 + r.2.2 * v.2.2) % 2
/-- Solutions of a two-row system: two planes. -/
def sols2 (r1 r2 : Nat × Nat × Nat) (t1 t2 : Nat) : Nat :=
  (gf2Vecs.filter (fun v => dot2 r1 v == t1 && dot2 r2 v == t2)).length
def row3 (n : Nat) : Nat × Nat × Nat := (n / 4 % 2, n / 2 % 2, n % 2)

set_option maxRecDepth 100000 in
/-- TWO AXES NEVER LOCK: no two-row system over GF(2), for any target, has exactly one solution. -/
theorem two_axes_never_lock :
  (List.range 64).all (fun m => (List.range 4).all (fun t =>
    sols2 (row3 (m / 8)) (row3 (m % 8)) (t / 2) (t % 2) != 1)) = true := by decide

set_option maxRecDepth 100000 in
/-- THREE AXES LOCK: of the 512 three-row systems, 168 have determinant one, each locking all eight targets. -/
theorem three_axes_lock :
  ((List.range 512).filter (fun m => det2 (row3 (m / 64)) (row3 (m / 8 % 8)) (row3 (m % 8)) == 1)).length = 168 ∧
  (List.range 512).all (fun m => det2 (row3 (m / 64)) (row3 (m / 8 % 8)) (row3 (m % 8)) != 1 ||
    gf2Vecs.all (fun t => (gf2Vecs.filter (fun v => dot2 (row3 (m / 64)) v == t.1 &&
      dot2 (row3 (m / 8 % 8)) v == t.2.1 && dot2 (row3 (m % 8)) v == t.2.2)).length == 1)) = true := by
  decide

```

```

/-- The quaternion units on integer coordinates. -/
structure Q4 where
  r : Int
  i : Int
  j : Int
  k : Int
  deriving DecidableEq
def qmul (a b : Q4) : Q4 :=
  (a.r*b.r - a.i*b.i - a.j*b.j - a.k*b.k,
   a.r*b.i + a.i*b.r + a.j*b.k - a.k*b.j,
   a.r*b.j - a.i*b.k + a.j*b.r + a.k*b.i,
   a.r*b.k + a.i*b.j - a.j*b.i + a.k*b.r)
/-- THE THIRD IS BEGOTTEN:  $i j = k$ ,  $i j k = -1$ , and  $i j \neq j i$ . -/
theorem third_begotten :
  qmul (0,1,0,0) (0,0,1,0) = (0,0,0,1) ∧ qmul (qmul (0,1,0,0) (0,0,1,0)) (0,0,0,1) = (-1,0,0,0) ∧
  qmul (0,1,0,0) (0,0,1,0) ≠ qmul (0,0,1,0) (0,1,0,0) := by decide

/-- Cayley–Dickson doubling on integer coordinates. -/
def cdconj : List Int → List Int
| [] => []
| a :: rest => a :: rest.map (fun x => -x)
def padd (a b : List Int) : List Int := List.zipWith (· + ·) a b
def psub (a b : List Int) : List Int := List.zipWith (· - ·) a b
def cdmul : Nat → List Int → List Int → List Int
| 0, _, _ => []
| _, +1, [a], [b] => [a * b]
| fuel + 1, a, b =>
  let n := a.length / 2
  psub (cdmul fuel (a.take n) (b.take n)) (cdmul fuel (cdconj (b.drop n)) (a.drop n)) ++
  padd (cdmul fuel (b.drop n) (a.take n)) (cdmul fuel (a.drop n) (cdconj (b.take n)))
def e (n i : Nat) : List Int := (List.range n).map (fun j => if j == i then 1 else 0)

set_option maxRecDepth 100000 in
/-- NO FOURTH, BY TWO WALLS: associativity fails at eight dimensions,  $[e1, e2, e4] = 2 e7$ , and division fails at
sixteen,  $(e1 + e10)(e5 + e14) = 0$  with both factors nonzero. -/
theorem no_fourth_axis :
  psub (cdmul 4 (cdmul 4 (e 8 1) (e 8 2)) (e 8 4)) (cdmul 4 (e 8 1) (cdmul 4 (e 8 2) (e 8 4)))
  = (e 8 7).map (fun x => 2 * x) ∧
  cdmul 5 (padd (e 16 1) (e 16 10)) (padd (e 16 5) (e 16 14)) = List.replicate 16 0 := by decide

/-- The rotations of the tetrahedron as even permutations of four vertices. -/
def invCount (σ : List Nat) : Nat :=
  let idx := List.range σ.length
  ((idx.flatMap (fun a => (idx.filter (fun b => a < b)).map (fun b => (a, b)))).filter
   (fun p => σ.getD p.1 0 > σ.getD p.2 0)).length
def perms24 : List (List Nat) :=
  [[0,1,2,3],[0,1,3,2],[0,2,1,3],[0,2,3,1],[0,3,1,2],[0,3,2,1],
   [1,0,2,3],[1,0,3,2],[1,2,0,3],[1,2,3,0],[1,3,0,2],[1,3,2,0],
   [2,0,1,3],[2,0,3,1],[2,1,0,3],[2,1,3,0],[2,3,0,1],[2,3,1,0],
   [3,0,1,2],[3,0,2,1],[3,1,0,2],[3,1,2,0],[3,2,0,1],[3,2,1,0]]
def a4 : List (List Nat) := perms24.filter (fun σ => invCount σ % 2 == 0)
def gates : List (Nat × Nat) :=
  [(0,1),(0,2),(0,3),(1,0),(1,2),(1,3),(2,0),(2,1),(2,3),(3,0),(3,1),(3,2)]

set_option maxRecDepth 100000 in
/-- TWELVE: twelve rotations, twelve directed gates, and exactly one rotation carries any gate to any gate. -/
theorem twelve_gates_one_torsor :
  a4.length = 12 ∧ gates.length = 12 ∧
  gates.all (fun g => gates.all (fun h =>
    (a4.filter (fun σ => (σ.getD g.1 0, σ.getD g.2 0) == h)).length == 1)) = true := by decide

/-- The eight sign patterns on three axes, total negation, and parity. -/
def cube : List (Bool × Bool × Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b => [false, true].map fun c => (a, b, c)
def neg3 (s : Bool × Bool × Bool) : Bool × Bool × Bool := (!s.1, !s.2.1, !s.2.2)
def par3 (s : Bool × Bool × Bool) : Nat :=
  (if s.1 then 1 else 0) + (if s.2.1 then 1 else 0) + (if s.2.2 then 1 else 0)
/-- The orientation of a sign pattern, the determinant of its diagonal: -1 to the number of flips. -/
def orient (s : Bool × Bool × Bool) : Int := if par3 s % 2 == 0 then 1 else -1

/-- EIGHT: eight patterns; the squared orientation, the lock scalar, reads one on all eight; total negation moves
every pattern, undoes itself, and joins each even pattern to an odd one. -/
theorem eight_patterns_four_orbits :
  cube.length = 8 ∧ cube.all (fun s => orient s * orient s == 1) = true ∧
  cube.all (fun s => neg3 s != s && neg3 (neg3 s) == s && orient (neg3 s) == -orient s) = true := by decide

/-- THE NINTH IS NOT A REFLECTION: no group of sign patterns, on any number of axes, has nine elements. -/
theorem no_reflection_group_of_nine : ∀ k : Nat, 2 ^ k ≠ 9
| 0, h => Nat.noConfusion (Nat.succ.inj h)
| k + 1, h =>
  have e : 2 ^ (k + 1) % 2 = 0 := Nat.mul_mod_left (2 ^ k) 2
  have e2 : 2 ^ (k + 1) % 2 = 9 % 2 := congrArg (· % 2) h

```

```

Nat.noConfusion (e.symm.trans e2 : 0 = 1)

/-- The ordered factor pairs of n, by first factor; the swap sends a to n / a. -/
def pairs (n : Nat) : List Nat := (List.range (n + 1)).filter (fun a => 0 < a && n % a == 0)
def seats (n : Nat) : Nat := ((pairs n).filter (fun a => a * a == n)).length

set_option maxRecDepth 100000 in
/-- THE NINTH IS THE SEAT: thirty-six has nine ordered factor pairs, four free orbits and one seat (6, 6). -/
theorem nine_is_eight_and_a_seat :
  (pairs 36).length = 9 ∧ seats 36 = 1 ∧ ((pairs 36).filter (fun a => a * a != 36)).length = 8 ∧
  (pairs 36).all (fun a => (pairs 36).contains (36 / a) && 36 / (36 / a) == a) = true := by decide

set_option maxHeartbeats 4000000 in
set_option maxRecDepth 200000 in
/-- ODD EXACTLY WHEN SEATED: for every n from 1 to 200 the fibre is odd exactly when it holds a seat, and it
never holds two. -/
theorem odd_fibre_iff_seat :
  (List.range 201).all (fun n => n == 0 ||
    ((pairs n).length % 2 == seats n % 2 && seats n ≤ 1)) = true := by decide

/-! ## V · The three legs on one orbit: forget, pay, supply -/

/-! ### Pay: the floor, from P1 and counting -/
def ND : List Nat → Prop
| [] => True
| x :: r => x ∉ r ∧ ND r
def AllLt (M : Nat) : List Nat → Prop
| [] => True
| x :: r => x < M ∧ AllLt M r

theorem allLt_mem (M : Nat) : ∀ (l : List Nat) (y : Nat), AllLt M l → y ∈ l → y < M
| [], _, _ => nomatch m
| x :: r, y, h, m => by
  cases m with
  | head => exact h.1
  | tail _ hm => exact allLt_mem M r y h.2 hm

theorem nd_map (h : Nat → Nat) : ∀ l : List Nat, ND l → (∀ a ∈ l, ∀ b ∈ l, h a = h b → a = b) → ND (l.map h)
| [], _, _ => trivial
| x :: r, hnd, hnj => by
  refine (fun m => ?_, nd_map h r hnd.2 (fun a ha b hb e => hnj a (List.Mem.tail x ha) b (List.Mem.tail x hb) e))
  obtain (y, hy, e) := List.mem_map.mp m
  have : y = x := hnj y (List.Mem.tail x hy) x (List.Mem.head r) e
  exact hnd.1 (this ▸ hy)

theorem allLt_map (h : Nat → Nat) (M : Nat) : ∀ l : List Nat, (∀ a ∈ l, h a < M) → AllLt M (l.map h)
| [], _ => trivial
| x :: r, hb => (hb x (List.Mem.head r), allLt_map h M r (fun a ha => hb a (List.Mem.tail x ha)))

/-- PIGEONHOLE: distinct numbers all below M number at most M. -/
theorem pigeonhole : ∀ (l : List Nat) (M : Nat), ND l → AllLt M l → l.length ≤ M
| [], _, _ => Nat.zero_le _
| x :: r, M, hnd, hlt => by
  have hx : x < M := hlt.1
  let sw : Nat → Nat := fun y => if y = M - 1 then x else y
  have hbound : ∀ a ∈ r, sw a < M - 1 := by
    intro a ha
    have haM := allLt_mem M r a hlt.2 ha
    have hax : a ≠ x := fun e => hnd.1 (e ▸ ha)
    show (if a = M - 1 then x else a) < M - 1
    by_cases e : a = M - 1
    · rw [if_pos e]
      have : x ≠ M - 1 := fun e2 => hax (e.trans e2.symm)
      omega
    · rw [if_neg e]; omega
  have hnj : ∀ a ∈ r, ∀ b ∈ r, sw a = sw b → a = b := by
    intro a ha b hb e
    have hax : a ≠ x := fun e2 => hnd.1 (e2 ▸ ha)
    have hbx : b ≠ x := fun e2 => hnd.1 (e2 ▸ hb)
    change (if a = M - 1 then x else a) = (if b = M - 1 then x else b) at e
    by_cases ea : a = M - 1 <; by_cases eb : b = M - 1
    · rw [ea, eb]
    · rw [if_pos ea, if_neg eb] at e; exact absurd e.symm hbx
    · rw [if_neg ea, if_pos eb] at e; exact absurd e hax
    · rw [if_neg ea, if_neg eb] at e; exact e
  have ih := pigeonhole (r.map sw) (M - 1) (nd_map sw r hnd.2 hnj) (allLt_map sw (M - 1) r hbound)
  rw [List.length_map] at ih
  show r.length + 1 ≤ M
  omega

/-- THE EXPORT. The whole was C × F; the step merges nothing (P1); the content halves. The freedom doubles. -/
theorem export_doubles (L : List Nat) (step : Nat → Nat) (C F C' F' : Nat) (hnd : ND L)
  (hwhole : L.length = C * F) (P1 : ∀ a ∈ L, ∀ b ∈ L, step a = step b → a = b)

```

```

(cells : ∀ a ∈ L, step a < C' * F') (hhalf : C = 2 * C') (hpos : 0 < C') : 2 * F ≤ F' := by
have hb := pigeonhole (L.map step) (C' * F') (nd_map step L hnd P1) (allLt_map step _ L cells)
rw [List.length_map, hwhole, hhalf, Nat.mul_assoc, Nat.mul_left_comm] at hb
exact Nat.le_of_mul_le_mul_left hb hpos

theorem one_bit_arrives (b b' : Nat) (h : 2 * 2 ^ b ≤ 2 ^ b') : b + 1 ≤ b' := by
refine Nat.lt_of_not_le fun hle => ?_
have h1 := Nat.pow_le_pow_right (by decide : 0 < 2) hle
have h2 := Nat.pow_pos (n := b) (by decide : 0 < 2)
omega

/-- The price of exported freedom in native units: temperature is energy per bit, so b bits cost b.T. -/
def price (T bits : Nat) : Nat := bits * T

/-- FROM COUNT TO HEAT. Under P1 a halving of the content exports at least one bit, which costs at least one T. -/
theorem count_to_heat (L : List Nat) (step : Nat → Nat) (C C' b b' T : Nat) (hnd : ND L)
(hwhole : L.length = C * 2 ^ b) (P1 : ∀ a ∈ L, ∀ x ∈ L, step a = step x → a = x)
(cells : ∀ a ∈ L, step a < C' * 2 ^ b') (hhalf : C = 2 * C') (hpos : 0 < C') :
b + 1 ≤ b' ∧ price T 1 ≤ price T (b' - b) := by
have hb := one_bit_arrives b b' (export_doubles L step C (2 ^ b) C' (2 ^ b') hnd hwhole P1 cells hhalf hpos)
exact (hb, Nat.mul_le_mul_right T (by omega))

/-- P1 IS LOAD-BEARING: a step that merges lets the content halve with no freedom exported. -/
theorem merge_exports_nothing :
[0, 1].length = 2 * 2 ^ 0 ∧ (∀ a ∈ [0, 1], (fun _ : Nat => 0) a < 1 * 2 ^ 0) ∧
~ (∀ a ∈ [0, 1], ∀ x ∈ [0, 1], (fun _ : Nat => (0 : Nat)) a = (fun _ : Nat => (0 : Nat)) x → a = x) := by
refine (rfl, ?, ?)
· intro a _; show 0 < 1 * 2 ^ 0; decide
· intro h; exact absurd (h 0 (List.Mem.head _)) 1 (List.Mem.tail _ (List.Mem.head _)) rfl (by decide)

/-- A finite configuration: the heights of its zeros on the line, and its off-line pairs, each an offset that is not
zero and a height. -/
structure FinCfg where
line : List Int
pairs : List (Int × Int)
offset_ne : ∀ q ∈ pairs, q.1 ≠ 0
def FinCfg.pts (F : FinCfg) : Config := fun p => (p.1 = 0 ∧ p.2 ∈ F.line) ∨ ∃ q ∈ F.pairs, p = q ∨ p = fold q
/-- The erased bits of a finite configuration: one per off-line pair, two points registered to one. -/
def erased (F : FinCfg) : Nat := F.pairs.length

theorem fincfg_closed (F : FinCfg) : Closed F.pts := fun p hp =>
match hp with
| Or.inl (hl, ht) => Or.inl (by show -p.1 = 0; rw [hl]; rfl, ht)
| Or.inr (q, hq, Or.inl e) => Or.inr (q, hq, Or.inr (congrArg fold e))
| Or.inr (q, hq, Or.inr e) => Or.inr (q, hq, Or.inl (by rw [e]; exact fold_involutive q))

/-- LEAST ERASURE IS ZERO ERASED BITS: "least" is a count. -/
theorem least_erasure_iff_zero_erased (F : FinCfg) : LeastErasure F.pts ↔ erased F = 0 := by
constructor
· intro h
unfold erased
cases hp : F.pairs with
| nil => rfl
| cons q r =>
have hq : q ∈ F.pairs := by rw [hp]; exact List.Mem.head r
exact absurd ((reg_erases_nothing_iff q).mp (h q (Or.inr (q, hq, Or.inl rfl)))) (F.offset_ne q hq)
· intro h p hp
have hnil : F.pairs = [] := List.eq_nil_of_length_eq_zero h
rcases hp with (hl, _) | (q, hq, _)
· exact (reg_erases_nothing_iff p).mpr hl
· rw [hnil] at hq; cases hq

/-- THE HEAT OF REGISTRATION IS ZERO EXACTLY AT THE VALUE: registering a finite configuration costs one T per
off-line pair, and it costs nothing exactly when every point stands on the line. -/
theorem price_zero_iff_value (F : FinCfg) (T : Nat) (hT : 0 < T) : price T (erased F) = 0 ↔ Value F.pts := by
constructor
· intro h p hp
have h0 : erased F = 0 := by
cases he : erased F with
| zero => rfl
| succ k => rw [he] at h; exact absurd h (Nat.pos_iff_ne_zero.mp (Nat.mul_pos (Nat.succ_pos k) hT))
exact (reg_erases_nothing_iff p).mp ((least_erasure_iff_zero_erased F).mpr h0 p hp)
· intro hv
have h0 := (least_erasure_iff_zero_erased F).mp (fun p hp => (reg_erases_nothing_iff p).mpr (hv p hp))
show erased F * T = 0
rw [h0, Nat.zero_mul]

/-- Each further off-line pair adds one T: the total is the erased bits times the floor. -/
theorem price_counts_pairs (F : FinCfg) (T : Nat) : price T (erased F) = F.pairs.length * T := rfl

/-- The measure of freedom: additivity taken over zero is dry. f (0 · n) = f 0 + f n forces f to vanish. -/
theorem additivity_over_zero_is_dry (f : Nat → Nat) (hmul : ∀ m n, f (m * n) = f m + f n) : ∀ n, f n = 0 := by
intro n

```

```

have h := hmul 0 n
rw [Nat.zero_mul] at h
omega

/-- THE LOG IS FORCED: additivity on positive arguments makes a measure linear on every tower; the unit is the
only choice, and fixing one bit at 2 makes it the bit count. -/
theorem linear_on_tower (f : Nat → Nat) (hmul : ∀ m n, 0 < m → 0 < n → f (m * n) = f m + f n)
  (v : Nat) (hv : 0 < v) : ∀ a, f (v ^ a) = a * f v
| 0 => by
  have h := hmul 1 1 (by decide) (by decide)
  show f 1 = 0 * f v
  rw [Nat.mul_one] at h; rw [Nat.zero_mul]; omega
| a + 1 => by
  rw [Nat.pow_succ, hmul _ _ (Nat.pow_pos (n := a) hv) hv, linear_on_tower f hmul v hv a, Nat.succ_mul]

theorem bits_forced (f : Nat → Nat) (hmul : ∀ m n, 0 < m → 0 < n → f (m * n) = f m + f n)
  (h2 : f 2 = 1) : ∀ a, f (2 ^ a) = a :=
  fun a => by rw [linear_on_tower f hmul 2 (by decide) a, h2, Nat.mul_one]

/-- The two-adic valuation: the count of the bit 2 in a number. -/
def v2 : Nat → Nat
| 0 => 0
| n + 1 => if (n + 1) % 2 = 0 then v2 ((n + 1) / 2) + 1 else 0
decreasing_by omega

theorem v2_mul_two (n : Nat) (hn : 0 < n) : v2 (2 * n) = v2 n + 1 := by
  obtain (k, rfl) : ∃ k, n = k + 1 := (n - 1, by omega)
  have e : 2 * (k + 1) = (2 * k + 1) + 1 := by omega
  rw [e, v2]
  have h1 : (2 * k + 1) % 2 = 0 := by omega
  have h2 : (2 * k + 1) / 2 = k + 1 := by omega
  rw [if_pos h1, h2]

theorem v2_odd (n : Nat) (h : n % 2 = 1) : v2 n = 0 := by
  cases n with
  | zero => exact absurd h (by decide)
  | succ k => rw [v2, if_neg (by omega)]

theorem v2_two : v2 2 = 1 := by
  have e := v2_mul_two 1 (by decide)
  rw [v2_odd 1 (by decide)] at e
  exact e

theorem v2_split : ∀ n, 0 < n → ∃ u, u % 2 = 1 ∧ n = 2 ^ v2 n * u := by
  intro n
  induction n using Nat.strongRecOn with
  | ind n ih =>
    intro hn
    by_cases hp : n % 2 = 0
    · obtain (m, rfl) : ∃ m, n = 2 * m := (n / 2, by omega)
      have hm : 0 < m := by omega
      obtain (u, hu, e) := ih m (by omega) hm
      refine (u, hu, ?_)
      rw [v2_mul_two m hm, Nat.pow_succ]
      calc 2 * m = 2 * (2 ^ v2 m * u) := by rw [← e]
      _ = 2 ^ v2 m * 2 * u := by rw [Nat.mul_comm 2, Nat.mul_assoc, Nat.mul_comm u 2, ← Nat.mul_assoc]
      · refine (n, by omega, ?_)
      rw [v2_odd n (by omega)]; simp

theorem v2_pow_odd (k u : Nat) (hu : u % 2 = 1) : v2 (2 ^ k * u) = k := by
  induction k with
  | zero => simp; exact v2_odd u hu
  | succ j ih =>
    have hpos : 0 < 2 ^ j * u := Nat.mul_pos (Nat.pow_pos (n := j) (by decide : 0 < 2)) (by omega)
    rw [Nat.pow_succ, Nat.mul_comm (2 ^ j) 2, Nat.mul_assoc, v2_mul_two _ hpos, ih]

/-- The two-adic valuation is additive on positive arguments. -/
theorem v2_additive (m n : Nat) (hm : 0 < m) (hn : 0 < n) : v2 (m * n) = v2 m + v2 n := by
  obtain (u, hu, em) := v2_split m hm
  obtain (w, hw, en) := v2_split n hn
  have huw : (u * w) % 2 = 1 := by rw [Nat.mul_mod, hu, hw]
  have e : m * n = 2 ^ (v2 m + v2 n) * (u * w) := by
    calc m * n = (2 ^ v2 m * u) * (2 ^ v2 n * w) := by rw [← em, ← en]
    _ = (2 ^ v2 m * 2 ^ v2 n) * (u * w) := by rw [Nat.mul_assoc, Nat.mul_left_comm u, ← Nat.mul_assoc]
    _ = 2 ^ (v2 m + v2 n) * (u * w) := by rw [Nat.pow_add]
  rw [e, v2_pow_odd _ _ huw]

/-! ### Supply: the wall, the calibration, the crossing -/
def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f (σ x) = f x

/-- THE WALL: an even reading never equals a target odd at a point. -/
theorem wall {α : Type} (σ : α → α) (f d : α → Bool) (x : α) (he : Even σ f) (ho : d (σ x) ≠ d x) : f ≠ d :=
  fun h => by subst h; exact ho (he x)

```

```

/-- THE CALIBRATION: one supplied odd witness fixes the bit, and fixes it uniquely. -/
theorem calibration {α : Type} (σ : α → α) (s d : α → Bool) (x : α) (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
  ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
  ∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c := by
  refine (xor (d x) (s x), {by cases s x <;> cases d x <;> rfl, by rw [hs, hd]; cases s x <;> cases d x <;> rfl}, ?_)
  intro c' hc; obtain (h1, -) := hc
  generalize hsx : s x = sv at h1; generalize hdx : d x = dv at h1
  cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd h1 (by decide)

/-- The executed frame: a formal content and the bit of whether it ran. Reading the content is formal. -/
abbrev Exec (Q : Type) := Bool × Q
def run {Q : Type} (s : Exec Q) : Exec Q := (!s.1, s.2)
def formalRead {Q : Type} (s : Exec Q) : Q := s.2
def ran {Q : Type} (s : Exec Q) : Bool := s.1

/-- EVERY FORMAL READING IS EVEN. -/
theorem formal_never_odd {Q : Type} (g : Q → Bool) : Even (run (Q := Q)) (fun s => g (formalRead s)) :=
  fun _ => rfl
/-- A CONSTANT SUPPLIES NOTHING. -/
theorem constant_no_crossing {Q : Type} (b : Bool) (s : Exec Q) : ¬ ((fun _ : Exec Q => b) (run s) ≠ b) :=
  fun h => h rfl
/-- THE CROSSING WALL: no reading of the formal content returns whether it ran. -/
theorem no_reading_returns_the_deed {Q : Type} (q : Q) : ∃ g : Q → Bool, ∀ s : Exec Q, g (formalRead s) = ran s :=
  fun (_, hg) => Bool.noConfusion ((hg (true, q)).symm.trans (hg (false, q)))

/-- THE THREE LEGS ON ONE ORBIT. The orbit {(1, t), (-1, t)}, the seat (0, t). FORGET: both members register to the
seat as one record. PAY: two distinct states sent injectively below F' need F' at least two. SUPPLY: for any
target odd on the orbit, one calibration bit against the side reconstructs it, and only one. -/
def side (p : Point) : Bool := decide (0 < p.1)
theorem three_legs_one_orbit (t : Int) (d : Point → Bool) (hd : d (fold (1, t)) = !d (1, t)) :
  (onLine (reg (1, t)) ∧ reg (fold (1, t)) = reg (1, t) ∧ fold (1, t) ≠ (1, t)) ∧
  (∀ (F' : Nat) (step : Point → Nat), step (1, t) ≠ step (fold (1, t)) →
    step (1, t) < F' → step (fold (1, t)) < F' → 2 ≤ F') ∧
  (∃ c : Bool, (d (1, t) = xor (side (1, t)) c ∧ d (fold (1, t)) = xor (side (fold (1, t))) c) ∧
    ∀ c', (d (1, t) = xor (side (1, t)) c' ∧ d (fold (1, t)) = xor (side (fold (1, t))) c') → c' = c) := by
  refine ((rfl, rfl, fun h => by cases (congrArg Prod.fst h), fun F' step hne h1 h2 => ?_, ?_))
  · match F', h1, h2 with
  | 0, h1, _ => exact absurd h1 (Nat.not_lt_zero _)
  | 1, h1, h2 => exact absurd ((Nat.le_zero.mp (Nat.le_of_lt_succ h1)).trans
    (Nat.le_zero.mp (Nat.le_of_lt_succ h2)).symm) hne
  | n + 2, _, _ => exact Nat.le_add_left 2 n
  · exact calibration fold side d (1, t) rfl hd

/-! ## VI · Primes: the Bridge on the multiplicative cut -/
def fib (n : Nat) : Nat := (pairs n).length
def isPrimeTrial (n : Nat) : Bool := 2 ≤ n && ((List.range n).filter (fun a => 2 ≤ a && n % a == 0)).isEmpty

set_option maxHeartbeats 4000000 in
set_option maxRecDepth 200000 in
/-- A PRIME IS ONE FREE ORBIT: for every n from 2 to 200, n is prime exactly when its ordered factor fibre has two
points, and then no seat. -/
theorem prime_iff_one_free_orbit :
  (List.range 201).all (fun n => n < 2 || ((isPrimeTrial n == (fib n == 2)) && (!isPrimeTrial n || seats n == 0))) = true := by
  decide

/-- The six pairs of the parity frame: a prime and a semiprime with equal residues mod 420. -/
def k4 : List (Nat × Nat) := [(11, 851), (13, 1273), (17, 437), (19, 2119), (23, 1703), (29, 869)]
def lg2 : Nat → Nat → Nat
| 0, _ => 0
| k + 1, n => if n < 2 then 0 else lg2 k (n / 2) + 1

set_option maxRecDepth 100000 in
/-- THE PARITY FRAME: residues mod 420 merge each pair; the bits paid by registering the product separate it, one
against two, and the parity of the bits paid is the sign: odd at the prime, even at the semiprime. -/
theorem parity_frame_paid_bits :
  k4.all (fun p => p.1 % 420 == p.2 % 420 && lg2 64 (fib p.1) == 1 && lg2 64 (fib p.2) == 2) = true := by
  decide

def divsN (n : Nat) : List Nat := (List.range (n + 1)).filter (fun d => 0 < d && n % d == 0)
def omegaIn (ps : List Nat) (d : Nat) : Nat := (ps.filter (fun p => d % p == 0)).length
def toggle (p d : Nat) : Nat := if d % p == 0 then d / p else d * p
def sig30 (d : Nat) : Bool × Bool × Bool := (d % 2 == 0, d % 3 == 0, d % 5 == 0)

set_option maxRecDepth 100000 in
/-- THE EIGHT ARE THE DIVISOR CUBE: the eight divisors of 30 are the eight sign patterns, one to one, and the
Möbius sign is the orientation of the pattern. -/
theorem divisor_cube_is_sign_cube :
  divsN 30 = [1, 2, 3, 5, 6, 10, 15, 30] ∧ ((divsN 30).map sig30).eraseDups.length = 8 ∧
  (divsN 30).all (fun d => orient (sig30 d) == (if omegaIn [2, 3, 5] d % 2 == 0 then 1 else -1)) = true := by
  decide

set_option maxRecDepth 200000 in

```

```

/-- EVERY FINITE FIBRE BALANCES: on the 32 divisors of 2·3·5·7·11 the toggle of 2 is a free involution that flips
the sign, so the signs balance sixteen against sixteen. -/
theorem fibre_balance :
  (divsN 2310).length = 32 ∧
  (divsN 2310).all (fun d => (divsN 2310).contains (toggle 2 d) && toggle 2 d != d &&
    toggle 2 (toggle 2 d) == d &&
    (omegaIn [2, 3, 5, 7, 11] d + omegaIn [2, 3, 5, 7, 11] (toggle 2 d)) % 2 == 1) = true ∧
  ((divsN 2310).filter (fun d => omegaIn [2, 3, 5, 7, 11] d % 2 == 0)).length = 16 := by decide

/-! ## VII · Wall, edge, seat -/

/-- The edge and the wall on the offset chart, d = 2 Re s - 1: the wall Re s = 1 is d = 1, the edge Re s = 0 is
d = -1, and the fold exchanges them; the seat d = 0 is their midpoint. -/
theorem wall_and_edge_exchanged (t : Int) : fold (1, t) = (-1, t) ∧ fold (-1, t) = (1, t) ∧ onLine (0, t) :=
  (rfl, rfl, rfl)

/-- Every prime's own mode p^d is unitary at exactly one depth: d = 0, the edge. -/
theorem prime_mode_unitary_iff (p d : Nat) (hp : 2 ≤ p) : p ^ d = 1 ↔ d = 0 := by
  constructor
  · intro h
    cases d with
    | zero => rfl
    | succ k =>
      have h1 : p ^ 1 ≤ p ^ (k + 1) := Nat.pow_le_pow_right (by omega) (by omega)
      rw [Nat.pow_one] at h1
      omega
  · intro h; rw [h]; rfl

/-- At s units of the floor per bit, the state 2^a weighs 2^(-a·s): s is the price per bit, and s = 1 is the
floor itself. -/
theorem price_per_bit_tower (a s : Nat) : (2 ^ a) ^ s = 2 ^ (a * s) := (Nat.pow_mul 2 a s).symm

theorem len_le_sum : ∀ l : List Nat, (∀ x ∈ l, 1 ≤ x) → l.length ≤ l.sum
| [], _ => Nat.le_refl 0
| x :: r, h => by
  have hx := h x (List.Mem.head r)
  have ih := len_le_sum r (fun y hy => h y (List.Mem.tail x hy))
  simp only [List.length_cons, List.sum_cons]
  omega

/-- THE WALL OF COUNTING: the block (2^j, 2^(j+1)] of shares one-in-n holds, in units of 1/2^(j+1), at least
2^j units, a half. Every block holds a half, so the shares of all states have no finite total at s = 1. -/
theorem wall_block_share (j : Nat) :
  2 ^ j ≤ ((List.range (2 ^ j)).map (fun i => 2 ^ (j + 1) / (2 ^ j + 1 + i))).sum := by
  have hlen : ((List.range (2 ^ j)).map (fun i => 2 ^ (j + 1) / (2 ^ j + 1 + i))).length = 2 ^ j := by simp
  have key := len_le_sum ((List.range (2 ^ j)).map (fun i => 2 ^ (j + 1) / (2 ^ j + 1 + i))) (by
    intro x hx
    obtain (i, hi, rfl) := List.mem_map.mp hx
    have hi' : i < 2 ^ j := List.mem_range.mp hi
    have hle : 2 ^ j + 1 + i ≤ 2 ^ (j + 1) := by rw [Nat.pow_succ]; omega
    exact (Nat.le_div_iff_mul_le (by omega)).mpr (by omega))
  rw [hlen] at key
  exact key

/-! ## VIII · The heat flow: the fold pulls every pair onto its seat -/

/-- Conjugation of the flow variable is the fold: with s = 1/2 + i z and Im z = y, the offset is d = -2y, and
y → -y sends d to -d. -/
theorem conjugation_is_the_fold (y : Int) : -2 * -y = -(-2 * y) := Int.mul_neg (-2) y

/-- The quadratic z^2 + b z + c under the flow c ↦ c - 2t. -/
def flow (t : Int) (q : Int × Int) : Int × Int := (q.1, q.2 - 2 * t)
theorem flow_inverse (t : Int) (q : Int × Int) : flow (-t) (flow t q) = q := by
  obtain (b, c) := q
  simp only [flow, Prod.mk.injEq, true_and]
  omega

/-- THE FLOW MERGES NOTHING: it is injective, so under the floor it exports nothing and costs nothing. -/
theorem flow_injective (t : Int) (p q : Int × Int) (h : flow t p = flow t q) : p = q := by
  rw [← flow_inverse t p, ← flow_inverse t q, h]

/-- The discriminant rises by exactly eight per unit of time. -/
theorem disc_flow (D c t : Int) : D - 4 * (c - 2 * t) = (D - 4 * c) + 8 * t := by omega

/-- The pull in squared form: the squared imaginary part of the pair of z^2 + c is c - 2t, falling by two per unit
of time, which is the partner's pull -1/b on b. -/
theorem pull_in_square (c t : Int) : c - 2 * (t + 1) = (c - 2 * t) - 2 := by omega

/-- Collision, doubled time u = 2t: a pair while u < c, the seat at u = c, two fixed points after. -/
theorem collision_sign (c u : Int) :
  (u < c → 4 * u - 4 * c < 0) ∧ (u = c → 4 * u - 4 * c = 0) ∧ (c < u → 0 < 4 * u - 4 * c) :=
  (fun _ => by omega, fun _ => by omega, fun _ => by omega)

/-- Zero slack: the double root on the seat leaves it under every backward step. -/
theorem zero_slack (t : Int) (ht : t < 0) : 0 - 4 * (0 - 2 * t) < 0 := by omega

/-- The only merge is the certificate: real-rootedness at a positive time takes one value on two states. -/
def realAt (t : Int) (q : Int × Int) : Bool := decide (0 ≤ q.1 * q.1 - 4 * (q.2 - 2 * t))
theorem certificate_merges : realAt 1 (0, 0) = realAt 1 (0, 1) ∧ ((0, 0) : Int × Int) ≠ (0, 1) ∧

```

```

    realAt 0 (0, 0) ≠ realAt 0 (0, 1) := by decide

/-! ## IX · The physical carrier -/

/-- Charge in thirds of the electron's charge; charge conjugation is negation; the carrier puts charge on the
offset, d = Q, at resolution twelve. -/
def conj (q : Int) : Int := -q
def carrier (q : Int) : Point := (q, 0)
/-- The carrier is equivariant, neutral lands on the line, and the electron and positron are one off-line pair of
one record. -/
theorem charge_carrier :
  (∀ q : Int, carrier (conj q) = fold (carrier q)) ∧ onLine (carrier 0) ∧ ¬ onLine (carrier (-3)) ∧
  fold (carrier (-3)) = carrier 3 ∧ reg (carrier 3) = reg (carrier (-3)) :=
  (fun _ => rfl, rfl, fun h => (by decide : ¬ ((-3 : Int) = 0)) h, rfl, rfl)

/-! ## X · Inhabitation: every conditional law has a model of its hypotheses -/
theorem value_from_the_act_inhabited : Nonempty ActualZeros :=
  ((onLine, fun p h => by show -p.1 = 0; rw [show p.1 = 0 from h]; rfl, fun p h => (reg_erases_nothing_iff p).mpr h))
theorem off_line_inhabited : ∃ p : Point, ¬ onLine p := ((1, 0), fun h => by cases h)
theorem weakest_forcing_premise_inhabited :
  ∃ P : Config → Prop, (∀ S, P S → Value S) ∧ (∀ S, Value S → P S) := (Value, fun _ h => h, fun _ h => h)
theorem keyless_forces_nothing_inhabited : ∃ Q : Config → Prop, ∀ S, Closed S → Q S :=
  (fun _ => True, fun _ => trivial)
theorem count_to_heat_inhabited :
  ND [0, 1] ∧ [0, 1].length = 2 * 2 ^ 0 ∧ (∀ a ∈ [0, 1], ∀ x ∈ [0, 1], (fun n : Nat => n) a = (fun n : Nat => n) x → a = x) ∧
  (∀ a ∈ [0, 1], (fun n : Nat => n) a < 1 * 2 ^ 1) ∧ 2 = 2 * 1 ∧ 0 < 1 := by
  refine ((by decide, by decide, trivial), rfl, fun a _ x _ h => h, ?_, rfl, by decide)
  intro a ha
  cases ha with
  | head => decide
  | tail _ h => cases h with
  | head => decide
  | tail _ h2 => cases h2
theorem bits_forced_inhabited :
  ∃ f : Nat → Nat, (∀ m n, 0 < m → 0 < n → f (m * n) = f m + f n) ∧ f 2 = 1 := (v2, v2_additive, v2_two)
theorem calibration_inhabited :
  (fun b : Bool => b) ((fun b : Bool => !b) true) = !((fun b : Bool => b) true) := rfl
theorem three_legs_inhabited (t : Int) : side (fold (1, t)) = !side (1, t) := rfl
theorem prime_mode_unitary_inhabited : 2 ≤ 2 := Nat.le_refl 2
theorem zero_slack_inhabited : (-1 : Int) < 0 := by decide
theorem record_carried_inhabited (S : Config) : SameRecord (Rec S) S ∧ Value (Rec S) :=
  (fun q => ((record_same S) q).symm, record_is_lossless S)
theorem price_zero_inhabited : 0 < 1 := Nat.succ_pos 0

/-! ## XI · The Bridge, whole -/

/-- What the Bridge says on no axiom at all. -/
def BridgeAtom : Prop :=
  (∀ p : Point, fold (fold p) = p) ∧ (∀ p : Point, fold p = p ↔ onLine p) ∧
  (∀ p : Point, onLine (reg p) ∧ (reg p).2 = p.2 ∧ reg (fold p) = reg p) ∧
  (∀ p : Point, ¬ onLine p → ¬ ∃ g : Point → Point, g (reg p) = p ∧ g (reg (fold p)) = fold p) ∧
  (∀ S : Config, LeastErasure S ↔ Value S) ∧
  (∀ Z : ActualZeros, Value Z.zeros) ∧
  (∀ (p : Point), ¬ onLine p → ∀ g : Config → Prop, (∀ S S', SameRecord S S' → (g S ↔ g S')) →
    ¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld p) ↔ Value (pairWorld p)))) ∧
  (∀ Q : Config → Prop, (∀ S, Closed S → Q S) → ¬ ∀ S, Closed S → Q S → Value S) ∧
  (∀ P : Config → Prop, (∀ S, P S → Value S) → (∀ S, Value S → P S) → ∀ S, P S ↔ Value S) ∧
  (∀ X : Frame, (∃ b : Carrier X, b.terminal = .bot) ↔ LineProperty X) ∧
  (¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v) ∧
  ((∀ b : Bool, twoPoint.τ (twoPoint.τ b) = b) ∧ (∀ b : Bool, twoPoint.τ b ≠ b) ∧ ¬ LineProperty twoPoint) ∧
  (∀ X : Frame, Nonempty (Supplied X) ↔ LineProperty X) ∧
  (∀ {Q : Type} ( _ : Q), ¬ ∃ g : Q → Bool, ∀ s : Exec Q, g (formalRead s) = ran s) ∧
  (∀ (t : Int) (d : Point → Bool), d (fold (1, t)) = !d (1, t) →
    (onLine (reg (1, t)) ∧ reg (fold (1, t)) = reg (1, t) ∧ fold (1, t) ≠ (1, t)) ∧
    (∀ (F' : Nat) (step : Point → Nat), step (1, t) ≠ step (fold (1, t)) →
      step (1, t) < F' → step (fold (1, t)) < F' → 2 ≤ F') ∧
    (∃ c : Bool, (d (1, t) = xor (side (1, t)) c ∧ d (fold (1, t)) = xor (side (fold (1, t))) c) ∧
      ∀ c', (d (1, t) = xor (side (1, t)) c' ∧ d (fold (1, t)) = xor (side (fold (1, t))) c') → c' = c)) ∧
  ((∀ q : Int, carrier (conj q) = fold (carrier q)) ∧ onLine (carrier 0) ∧ ¬ onLine (carrier (-3)) ∧
    fold (carrier (-3)) = carrier 3 ∧ reg (carrier 3) = reg (carrier (-3))) ∧
  (List.range 64).all (fun m => (List.range 4).all (fun t =>
    sols2 (row3 (m % 8)) (t / 2) (t % 2) != 1)) = true ∧
  a4.length = 12 ∧ cube.length = 8 ∧ (pairs 36).length = 9 ∧ seats 36 = 1 ∧
  (List.range 201).all (fun n => n == 0 || ((pairs n).length % 2 == seats n % 2 && seats n ≤ 1)) = true ∧
  (List.range 201).all (fun n => n < 2 || ((isPrimeTrial n == (fib n == 2)) && (!isPrimeTrial n || seats n == 0))) = true ∧
  k4.all (fun p => p.1 % 420 == p.2 % 420 && lg2 64 (fib p.1) == 1 && lg2 64 (fib p.2) == 2) = true ∧
  ((divsN 2310).filter (fun d => omegaIn [2, 3, 5, 7, 11] d % 2 == 0)).length = 16 ∧
  (realAt 1 (0, 0) = realAt 1 (0, 1) ∧ ((0, 0) : Int × Int) ≠ (0, 1) ∧ realAt 0 (0, 0) ≠ realAt 0 (0, 1)) ∧
  (∀ S : Config, Extremal S ↔ Value S) ∧
  (∀ S S' : Config, SameRecord S' S → Value S' → ∀ q, S' q ↔ Rec S q) ∧
  (∀ F : FinCfg, LeastErasure F.pts ↔ erased F = 0) ∧
  (∀ (F : FinCfg) (T : Nat), 0 < T → (price T (erased F) = 0 ↔ Value F.pts))

```

```

/-- THE ATOM OF THE BRIDGE, ON NO AXIOM: the fold and its line; the registration; the lost bit; least erasure as
the value; the act; the record, the keyless premise and the weakest premise; the carrier, its irreversibility,
the coherent denial and the socket; the deed no reading returns; the three legs on one orbit; the charge
carrier; two axes never lock; twelve, eight, and nine as eight and a seat; odd exactly when seated; a prime as
one free orbit; the parity frame paid in bits; the balance of a fibre; the certificate's merge; leastness in the
fibre as the value; the record carried uniquely; least erasure as zero erased bits; and the heat of registration
zero exactly at the value. -/
theorem the_bridge_atom : BridgeAtom :=
  (fold_involutive, the_cut_is_the_line, fun p => (reg_lands p, reg_keeps_height p, reg_forgets_side p),
  one_bit_lost, least_erasure_iff_value, value_from_the_act, record_decides_nothing, keyless_forces_nothing,
  weakest_forcing_premise, halted_iff, cannot_reverse, denial_is_coherent, socket_is_the_property,
  fun q => no_reading_returns_the_deed q, three_legs_one_orbit, charge_carrier, two_axes_never_lock,
  twelve_gates_one_torsor.1, eight_patterns_four_orbits.1, nine_is_eight_and_a_seat.1,
  nine_is_eight_and_a_seat.2.1, odd_fibre_iff_seat, prime_iff_one_free_orbit, parity_frame_paid_bits,
  fibre_balance.2.2, certificate_merges, extremal_iff_value, record_carried_uniquely,
  least_erasure_iff_zero_erased, price_zero_iff_value)

/-- THE BRIDGE FROM FIRST PRINCIPLE: the atom, and on the standard axioms the rest. The line property is keyed and
the fold law keyless, and every frame property is one or the other; no ninth reflection on any number of
axes; under P1 a halving exports a bit priced at one T, and P1 is load-bearing; additivity over zero is dry and
on positive arguments forces the bit count, with the two-adic valuation as its model; every prime mode is
unitary only at the edge; the counting wall; the price per bit; the flow merges nothing, collides on the seat,
pulls by two per unit time and leaves the seat backward; conjugation is the fold. -/
theorem the_bridge_from_first_principle :
  BridgeAtom ∧
  (Keyed LineProperty ∧ ¬ Keyless LineProperty) ∧ Keyless (fun X => ∀ s, X.τ (X.τ s) = s) ∧
  (∀ Q : Frame → Prop, (Keyless Q ∨ Keyed Q) ∧ ¬ (Keyless Q ∧ Keyed Q)) ∧
  (∀ k : Nat, 2 ^ k ≠ 9) ∧
  (∀ (L : List Nat) (step : Nat → Nat) (C C' b b' T : Nat), ND L → L.length = C * 2 ^ b →
    (∀ a ∈ L, ∀ x ∈ L, step a = step x → a = x) → (∀ a ∈ L, step a < C' * 2 ^ b') → C = 2 * C' → 0 < C' →
    b + 1 ≤ b' ∧ price T 1 ≤ price T (b' - b)) ∧
  ¬ (∀ a ∈ [0, 1], ∀ x ∈ [0, 1], (fun _ : Nat => (0 : Nat)) a = (fun _ : Nat => (0 : Nat)) x → a = x) ∧
  (∀ f : Nat → Nat, (∀ m n, f (m * n) = f m + f n) → ∀ n, f n = 0) ∧
  (∀ f : Nat → Nat, (∀ m n, 0 < m → 0 < n → f (m * n) = f m + f n) → f 2 = 1 → ∀ a, f (2 ^ a) = a) ∧
  (∃ f : Nat → Nat, (∀ m n, 0 < m → 0 < n → f (m * n) = f m + f n) ∧ f 2 = 1) ∧
  (∀ p d : Nat, 2 ≤ p → (p ^ d = 1 ↔ d = 0)) ∧
  (∀ a s : Nat, (2 ^ a) ^ s = 2 ^ (a * s)) ∧
  (∀ j : Nat, 2 ^ j ≤ ((List.range (2 ^ j)).map (fun i => 2 ^ (j + 1) / (2 ^ j + 1 + i))).sum) ∧
  (∀ (t : Int) (p q : Int × Int), flow t p = flow t q → p = q) ∧
  (∀ c u : Int, (u < c → 4 * u - 4 * c < 0) ∧ (u = c → 4 * u - 4 * c = 0) ∧ (c < u → 0 < 4 * u - 4 * c)) ∧
  (∀ c t : Int, c - 2 * (t + 1) = (c - 2 * t) - 2) ∧
  (∀ t : Int, t < 0 → 0 - 4 * (0 - 2 * t) < 0) ∧
  (∀ y : Int, -2 * -y = -(2 * y)) :=
  (the_bridge_atom, line_property_is_keyed, symmetry_is_keyless, discriminator, no_reflection_group_of_nine,
  count_to_heat, merge_exports_nothing.2.2, additivity_over_zero_is_dry, bits_forced, bits_forced_inhabited,
  prime_mode_unitary_iff, price_per_bit_tower, wall_block_share, flow_injective, collision_sign, pull_in_square,
  zero_slack, conjugation_is_the_fold)

end BFP

/-! ## The cones, pinned. Each line is the compiler's own; a drifted cone fails the compile. -/
/-- info: 'BFP.neg_neg_free' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.neg_neg_free
/-- info: 'BFP.neg_self_zero' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.neg_self_zero
/-- info: 'BFP.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.fold_involutive
/-- info: 'BFP.the_cut_is_the_line' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.the_cut_is_the_line
/-- info: 'BFP.reg_lands' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reg_lands
/-- info: 'BFP.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reg_keeps_height
/-- info: 'BFP.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reg_forgets_side
/-- info: 'BFP.reg_erases_nothing_iff' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reg_erases_nothing_iff
/-- info: 'BFP.off_line_pair' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.off_line_pair
/-- info: 'BFP.one_bit_lost' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.one_bit_lost
/-- info: 'BFP.least_erasure_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.least_erasure_iff_value
/-- info: 'BFP.nothing_escapes_least_erasure' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.nothing_escapes_least_erasure
/-- info: 'BFP.pair_world_closed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.pair_world_closed
/-- info: 'BFP.registered_closed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.registered_closed
/-- info: 'BFP.one_record' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.one_record
/-- info: 'BFP.registered_has_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.registered_has_value

```

```

/-- info: 'BFP.pair_world_lacks_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.pair_world_lacks_value
/-- info: 'BFP.record_decides_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_decides_nothing
/-- info: 'BFP.keyless_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.keyless_forces_nothing
/-- info: 'BFP.weakest_forcing_premise' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.weakest_forcing_premise
/-- info: 'BFP.value_from_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.value_from_the_act
/-- info: 'BFP.reader_frame' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reader_frame
/-- info: 'BFP.record_is_lossless' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_is_lossless
/-- info: 'BFP.record_same' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_same
/-- info: 'BFP.extremal_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.extremal_iff_value
/-- info: 'BFP.record_carried_uniquely' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_carried_uniquely
/-- info: 'BFP.chart_line_property' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.chart_line_property
/-- info: 'BFP.cannot_lie' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.cannot_lie
/-- info: 'BFP.cannot_deviate' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.cannot_deviate
/-- info: 'BFP.halted_iff' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.halted_iff
/-- info: 'BFP.cannot_reverse' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.cannot_reverse
/-- info: 'BFP.cannot_extend' depends on axioms: [Quot.sound] -/
#guard_msgs in #print axioms BFP.cannot_extend
/-- info: 'BFP.denial_is_coherent' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.denial_is_coherent
/-- info: 'BFP.discriminator' depends on axioms: [propxt, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BFP.discriminator
/-- info: 'BFP.symmetry_is_keyless' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.symmetry_is_keyless
/-- info: 'BFP.line_property_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.line_property_is_keyed
/-- info: 'BFP.socket_is_the_property' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.socket_is_the_property
/-- info: 'BFP.no_socket_off_line' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.no_socket_off_line
/-- info: 'BFP.act_seats_the_carrier' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.act_seats_the_carrier
/-- info: 'BFP.two_axes_never_lock' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.two_axes_never_lock
/-- info: 'BFP.three_axes_lock' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.three_axes_lock
/-- info: 'BFP.third_begotten' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.third_begotten
/-- info: 'BFP.no_fourth_axis' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.no_fourth_axis
/-- info: 'BFP.twelve_gates_one_torsor' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.twelve_gates_one_torsor
/-- info: 'BFP.eight_patterns_four_orbits' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.eight_patterns_four_orbits
/-- info: 'BFP.no_reflection_group_of_nine' depends on axioms: [propxt] -/
#guard_msgs in #print axioms BFP.no_reflection_group_of_nine
/-- info: 'BFP.nine_is_eight_and_a_seat' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.nine_is_eight_and_a_seat
/-- info: 'BFP.odd_fibre_iff_seat' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.odd_fibre_iff_seat
/-- info: 'BFP.allLt_mem' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.allLt_mem
/-- info: 'BFP.nd_map' depends on axioms: [propxt, Quot.sound] -/
#guard_msgs in #print axioms BFP.nd_map
/-- info: 'BFP.allLt_map' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.allLt_map
/-- info: 'BFP.pigeonhole' depends on axioms: [propxt, Quot.sound] -/
#guard_msgs in #print axioms BFP.pigeonhole
/-- info: 'BFP.export_doubles' depends on axioms: [propxt, Quot.sound] -/
#guard_msgs in #print axioms BFP.export_doubles
/-- info: 'BFP.one_bit_arrives' depends on axioms: [propxt, Quot.sound] -/
#guard_msgs in #print axioms BFP.one_bit_arrives
/-- info: 'BFP.count_to_heat' depends on axioms: [propxt, Quot.sound] -/
#guard_msgs in #print axioms BFP.count_to_heat
/-- info: 'BFP.merge_exports_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.merge_exports_nothing
/-- info: 'BFP.fincfg_closed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.fincfg_closed
/-- info: 'BFP.least_erasure_iff_zero_erased' does not depend on any axioms -/

```

```

#guard_msgs in #print axioms BFP.least_erasure_iff_zero_erased
/-- info: 'BFP.price_zero_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.price_zero_iff_value
/-- info: 'BFP.price_counts_pairs' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.price_counts_pairs
/-- info: 'BFP.additivity_over_zero_is_dry' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.additivity_over_zero_is_dry
/-- info: 'BFP.linear_on_tower' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.linear_on_tower
/-- info: 'BFP.bits_forced' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.bits_forced
/-- info: 'BFP.v2_mul_two' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_mul_two
/-- info: 'BFP.v2_odd' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_odd
/-- info: 'BFP.v2_two' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_two
/-- info: 'BFP.v2_split' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_split
/-- info: 'BFP.v2_pow_odd' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_pow_odd
/-- info: 'BFP.v2_additive' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_additive
/-- info: 'BFP.wall' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.wall
/-- info: 'BFP.calibration' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.calibration
/-- info: 'BFP.formal_never_odd' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.formal_never_odd
/-- info: 'BFP.constant_no_crossing' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.constant_no_crossing
/-- info: 'BFP.no_reading_returns_the_deed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.no_reading_returns_the_deed
/-- info: 'BFP.three_legs_one_orbit' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.three_legs_one_orbit
/-- info: 'BFP.prime_iff_one_free_orbit' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.prime_iff_one_free_orbit
/-- info: 'BFP.parity_frame_paid_bits' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.parity_frame_paid_bits
/-- info: 'BFP.divisor_cube_is_sign_cube' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.divisor_cube_is_sign_cube
/-- info: 'BFP.fibre_balance' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.fibre_balance
/-- info: 'BFP.wall_and_edge_exchanged' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.wall_and_edge_exchanged
/-- info: 'BFP.prime_mode_unitary_iff' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.prime_mode_unitary_iff
/-- info: 'BFP.price_per_bit_tower' depends on axioms: [propext] -/
#guard_msgs in #print axioms BFP.price_per_bit_tower
/-- info: 'BFP.len_le_sum' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.len_le_sum
/-- info: 'BFP.wall_block_share' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.wall_block_share
/-- info: 'BFP.conjugation_is_the_fold' depends on axioms: [propext] -/
#guard_msgs in #print axioms BFP.conjugation_is_the_fold
/-- info: 'BFP.flow_inverse' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.flow_inverse
/-- info: 'BFP.flow_injective' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.flow_injective
/-- info: 'BFP.disc_flow' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.disc_flow
/-- info: 'BFP.pull_in_square' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.pull_in_square
/-- info: 'BFP.collision_sign' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.collision_sign
/-- info: 'BFP.zero_slack' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.zero_slack
/-- info: 'BFP.certificate_merges' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.certificate_merges
/-- info: 'BFP.charge_carrier' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.charge_carrier
/-- info: 'BFP.value_from_the_act_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.value_from_the_act_inhabited
/-- info: 'BFP.off_line_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.off_line_inhabited
/-- info: 'BFP.weakest_forcing_premise_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.weakest_forcing_premise_inhabited
/-- info: 'BFP.keyless_forces_nothing_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.keyless_forces_nothing_inhabited
/-- info: 'BFP.count_to_heat_inhabited' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.count_to_heat_inhabited
/-- info: 'BFP.bits_forced_inhabited' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.bits_forced_inhabited

```

```

/-- info: 'BFP.calibration_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.calibration_inhabited
/-- info: 'BFP.three_legs_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.three_legs_inhabited
/-- info: 'BFP.prime_mode_unitary_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.prime_mode_unitary_inhabited
/-- info: 'BFP.zero_slack_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.zero_slack_inhabited
/-- info: 'BFP.record_carried_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_carried_inhabited
/-- info: 'BFP.price_zero_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.price_zero_inhabited
/-- info: 'BFP.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.the_bridge_atom
/-- info: 'BFP.the_bridge_from_first_principle' depends on axioms: [propxet, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BFP.the_bridge_from_first_principle

#eval "BRIDGE FROM FIRST PRINCIPLE · the end of the file was reached"

```

The twin. Bridge_Twin.f90

```

! Bridge_Twin.f90 . the executed twin of Bridge_From_First_Principle.lean.
! Every check is a finite computation that ends. No named human result is a premise.
! Build (sealed): gfortran -std=f2018 -O2 -fno-fast-math -ffp-contract=off -Wall -Wextra
program bridge_twin
  use, intrinsic :: iso_fortran_env, only: dp => real64, i8 => int64
  implicit none
  integer :: nchk = 0, nfail = 0
  call conduct()
  call atom()
  call three_twelve_eight()
  call pay()
  call supply()
  call primes()
  call wall_edge_seat()
  call heat_flow()
  write(*,'(a,i0,a,i0,a)') 'BRIDGE-TWIN-JSON: {"checks":', nchk, ', "failures":', nfail, '}'
  if (nfail /= 0) error stop 1
contains
  subroutine check(name, ok)
    character(*), intent(in) :: name
    logical, intent(in) :: ok
    nchk = nchk + 1
    if (.not. ok) nfail = nfail + 1
    write(*,'(a,a)') merge(' PASS ', ' FAIL ', ok), name
  end subroutine check

  ! 0 . conduct: the arithmetic this twin runs on is the arithmetic it claims
  subroutine conduct()
    real(dp), volatile :: one, x, y, r, z
    one = 1.0_dp
    x = one + scale(one, -30); y = one - scale(one, -30)
    r = x*y - one
    z = ieee_nan()
    write(*,'(a)') '0 CONDUCT'
    call check('0: no fused multiply-add executed (a fused path reads -2^60)', abs(r) <= 0.0_dp)
    call check('0: NaN is ordered against nothing, itself included', .not. (z <= z .or. z > z))
  end subroutine conduct
  function ieee_nan() result(z)
    use, intrinsic :: ieee_arithmetic, only: ieee_value, ieee_quiet_nan
    real(dp) :: z
    z = ieee_value(1.0_dp, ieee_quiet_nan)
  end function ieee_nan

  ! I . the atom on a window: fold, line, registration, the lost bit, least erasure equal to the value
  pure function foldp(p) result(q)
    integer, intent(in) :: p(2)
    integer :: q(2)
    q = [-p(1), p(2)]
  end function foldp
  pure function regp(p) result(q)
    integer, intent(in) :: p(2)
    integer :: q(2)
    q = [0, p(2)]
  end function regp
  subroutine atom()
    integer :: d, t, k, m, g1, g2, g3, h, nboth, none_
    integer :: g(0:2), pd(9), pt(9), p(2)
    logical :: ok1, ok2, ok3, le, va
    write(*,'(a)') 'I THE ATOM'
    ok1 = .true.; ok2 = .true.; ok3 = .true.
    do d = -50, 50
      do t = -5, 5

```

```

    p = [d, t]
    if (any(foldp(foldp(p)) /= p)) ok1 = .false.
    if (all(foldp(p) == p) .neqv. (d == 0)) ok1 = .false.
    if (all(regp(p) == p) .neqv. (d == 0)) ok2 = .false.
    if (any(regp(p) /= [0, t])) ok2 = .false.
    if (d /= 0) then
        if (all(foldp(p) == p)) ok3 = .false.
        if (any(regp(foldp(p)) /= regp(p))) ok3 = .false.
    end if
end do
end do
call check('I: the fold undoes itself and fixes exactly the line, 1111 points', ok1)
call check('I: the registration lands on the line, keeps the height, erases nothing exactly on it', ok2)
call check('I: every off-line point has a distinct partner of one record, 1100 points', ok3)
! window: d in {-1,0,1}, t in {0,1,2}; the records are the three heights; 9^3 = 729 maps from records to points
k = 0
do t = 0, 2
    do d = -1, 1
        k = k + 1; pd(k) = d; pt(k) = t
    end do
end do
nboth = 0; none_ = 0
do g1 = 1, 9
    do g2 = 1, 9
        do g3 = 1, 9
            g = [g1, g2, g3]
            do h = 0, 2
                ! the pair at height h is (1,h) and (-1,h), and both register to the record of height h
                m = g(h)
                if (pt(m) == h .and. abs(pd(m)) == 1) none_ = none_ + 1
                if (pt(m) == h .and. pd(m) == 1 .and. pt(m) == h .and. pd(m) == -1) nboth = nboth + 1
            end do
        end do
    end do
end do
write(*, '(a,i0,a,i0)') ' 729 maps from records to points: returning both partners ', nboth, &
    ' ; returning one member ', none_
call check('I: no map from records returns both partners of a pair', nboth == 0 .and. none_ == 486)
! least erasure, read through the registration, against the value, read through the offset
ok1 = .true.
do m = 0, 4095
    le = .true.; va = .true.
    do k = 0, 11
        if (btest(m, k)) then
            p = [mod(k, 3) - 1, k / 3]
            if (any(regp(p) /= p)) le = .false.
            if (p(1) /= 0) va = .false.
        end if
    end do
    if (le .neqv. va) ok1 = .false.
end do
call check('I: least erasure equals the value on all 4096 subsets of twelve points', ok1)
end subroutine atom

! II . three, twelve, eight, the ninth
subroutine three_twelve_eight()
    integer :: m, t, v, cnt, nlock, a, b, c, i, j, n, nperm, ninv, s, ok_cnt
    integer :: r(3,3), perm(4), even_perms(4,12)
    logical :: ok, used(4)
    write(*, '(a)') 'II THREE, TWELVE, EIGHT, THE NINTH'
    ok = .true.
    do m = 0, 63
        do t = 0, 3
            cnt = 0
            do v = 0, 7
                if (mod(rowdot(m/8, v), 2) == t/2 .and. mod(rowdot(mod(m,8), v), 2) == mod(t,2)) cnt = cnt + 1
            end do
            if (cnt == 1) ok = .false.
        end do
    end do
    call check('II: no two-axis system over GF(2) has exactly one solution, 256 systems', ok)
    nlock = 0; ok = .true.
    do m = 0, 511
        do i = 1, 3
            r(i,1) = mod(ishft(m, -(3*(3-i)+2)), 2); r(i,2) = mod(ishft(m, -(3*(3-i)+1)), 2)
            r(i,3) = mod(ishft(m, -(3*(3-i))), 2)
        end do
        if (det2(r) == 1) then
            nlock = nlock + 1
            do t = 0, 7
                cnt = 0
                do v = 0, 7
                    if (mod(r(1,1)*bit(v,2)+r(1,2)*bit(v,1)+r(1,3)*bit(v,0), 2) == bit(t,2) .and. &

```

```

        mod(r(2,1)*bit(v,2)+r(2,2)*bit(v,1)+r(2,3)*bit(v,0),2) == bit(t,1) .and. &
        mod(r(3,1)*bit(v,2)+r(3,2)*bit(v,1)+r(3,3)*bit(v,0),2) == bit(t,0)) cnt = cnt + 1
    end do
    if (cnt /= 1) ok = .false.
end do
end if
end do
call check('II: 168 three-axis systems have determinant one and each locks all eight targets', nlock == 168 .and. ok)
! twelve: even permutations of four, acting on the twelve directed gates
nperm = 0
do a = 1, 4
    do b = 1, 4
        do c = 1, 4
            do s = 1, 4
                perm = [a, b, c, s]
                used = .false.; ok = .true.
                do i = 1, 4
                    if (used(perm(i))) ok = .false.
                    used(perm(i)) = .true.
                end do
                if (.not. ok) cycle
                ninv = 0
                do i = 1, 3
                    do j = i+1, 4
                        if (perm(i) > perm(j)) ninv = ninv + 1
                    end do
                end do
                if (mod(ninv, 2) == 0) then
                    nperm = nperm + 1; even_perms(:, nperm) = perm
                end if
            end do
        end do
    end do
end do
ok = (nperm == 12)
do a = 1, 4
    do b = 1, 4
        if (a == b) cycle
        do c = 1, 4
            do s = 1, 4
                if (c == s) cycle
                ok_cnt = 0
                do n = 1, nperm
                    if (even_perms(a, n) == c .and. even_perms(b, n) == s) ok_cnt = ok_cnt + 1
                end do
                if (ok_cnt /= 1) ok = .false.
            end do
        end do
    end do
end do
call check('II: twelve rotations, and exactly one carries any directed gate to any directed gate', ok)
! eight: sign patterns, total negation free, orientation flipped
ok = .true.
do m = 0, 7
    if (ieor(m, 7) == m) ok = .false.
    if (mod(popcnt(m) + popcnt(ieor(m, 7)), 2) /= 1) ok = .false.
end do
call check('II: eight patterns, total negation moves each and joins an even to an odd', ok)
! the ninth: 2^k is never 9, and every odd fibre holds exactly one seat
ok = .true.
do i = 0, 62
    if (2_i8*i == 9_i8) ok = .false.
end do
call check('II: 2^k is never nine, k = 0..62', ok)
end subroutine three_twelve_eight
integer function rowdot(rr, v)
    integer, intent(in) :: rr, v
    rowdot = bit(rr,2)*bit(v,2) + bit(rr,1)*bit(v,1) + bit(rr,0)*bit(v,0)
end function rowdot
integer function bit(x, k)
    integer, intent(in) :: x, k
    bit = mod(ishft(x, -k), 2)
end function bit
integer function det2(r)
    integer, intent(in) :: r(3,3)
    det2 = mod(r(1,1)*(r(2,2)*r(3,3)+r(2,3)*r(3,2)) + r(1,2)*(r(2,1)*r(3,3)+r(2,3)*r(3,1)) + &
        r(1,3)*(r(2,1)*r(3,2)+r(2,2)*r(3,1)), 2)
end function det2

! III . pay: pigeonhole and the export, exhaustively on small wholes
logical function injective_exists(n, m)
    integer, intent(in) :: n, m
    integer :: f(8), i, j

```

```

logical :: inj
integer(i8) :: code, total, cc
injective_exists = .false.
total = int(m, i8)**n
do code = 0_i8, total - 1_i8
  cc = code
  do i = 1, n
    f(i) = int(mod(cc, int(m, i8))); cc = cc / int(m, i8)
  end do
  inj = .true.
  do i = 1, n - 1
    do j = i + 1, n
      if (f(i) == f(j)) inj = .false.
    end do
  end do
  if (inj) then
    injective_exists = .true.; return
  end if
end do
end function injective_exists
subroutine pay()
  integer :: n, m, cp, f, fp, k
  integer, parameter :: cps(5) = [1, 1, 1, 2, 3], fs(5) = [1, 2, 3, 1, 1], fpmx(5) = [10, 8, 8, 8, 3]
  logical :: ok
  integer(i8) :: a, b
  write(*, '(a)') 'III PAY'
  ok = .true.
  do n = 1, 6
    do m = 1, 6
      if (injective_exists(n, m) .neqv. (n <= m)) ok = .false.
    end do
  end do
  call check('III: n distinct states fit injectively in m cells exactly when n <= m, every n, m <= 6', ok)
  ok = .true.
  do k = 1, 5
    cp = cps(k); f = fs(k)
    do fp = 1, fpmx(k)
      n = 2*cp*f; m = cp*fp
      if (injective_exists(n, m) .neqv. (2*f <= fp)) ok = .false.
    end do
  end do
  call check('III: the content halves and nothing merges exactly when the freedom at least doubles', ok)
  call check('III: a merging step halves the content with no freedom exported (two states, one cell)', &
    .not. injective_exists(2, 1))
  ok = .true.
  do a = 1_i8, 300_i8
    do b = 1_i8, 300_i8
      if (v2(a*b) /= v2(a) + v2(b)) ok = .false.
    end do
  end do
  call check('III: the two-adic valuation is additive on positive arguments, 90000 pairs', ok .and. v2(2_i8) == 1)
end subroutine pay
integer function v2(x)
  integer(i8), intent(in) :: x
  integer(i8) :: y
  v2 = 0; y = x
  do while (mod(y, 2_i8) == 0_i8)
    v2 = v2 + 1; y = y / 2_i8
  end do
end function v2

! IV . supply: the calibration is unique, and no reading of the content returns the deed
subroutine supply()
  integer :: sx, dx, c, ncal, gcode, q, nret
  logical :: ok, all_match
  write(*, '(a)') 'IV SUPPLY'
  ok = .true.
  do sx = 0, 1
    do dx = 0, 1
      ncal = 0
      do c = 0, 1
        ! on the orbit {x, sigma x}: d(x) = s(x) xor c and d(sigma x) = s(sigma x) xor c, with s, d odd
        if (dx == ieor(sx, c) .and. 1 - dx == ieor(1 - sx, c)) ncal = ncal + 1
      end do
      if (ncal /= 1) ok = .false.
    end do
  end do
  call check('IV: for every odd witness and odd target, exactly one calibration bit', ok)
  nret = 0
  do gcode = 0, 7
    all_match = .true.
    do q = 0, 2
      ! a reading of the content q must equal ran on the executed state (1, q) and on the unexecuted (0, q)
    end do
  end do
end subroutine supply

```

```

        if (bit(gcode, q) /= 1) all_match = .false.
        if (bit(gcode, q) /= 0) all_match = .false.
    end do
    if (all_match) nret = nret + 1
end do
call check('IV: of the 8 readings of a three-element content, none returns whether it ran', nret == 0)
end subroutine supply

! V . primes: one free orbit, the parity frame, the divisor cube, the balance
subroutine primes()
integer, parameter :: N = 1000000
integer, allocatable :: dc(:), acc(:), mu(:)
logical, allocatable :: comp(:)
integer :: d, m, r, k, nsq
logical :: ok_odd, ok_bal, ok_prime, ok_k4, ok_seat
integer, parameter :: kp(6) = [11, 13, 17, 19, 23, 29], ks(6) = [851, 1273, 437, 2119, 1703, 869]
write(*,'(a)') 'V PRIMES'
allocate(dc(N), acc(N), mu(N), comp(N))
dc = 0; acc = 0; mu = 1; comp = .false.
do d = 2, N
    if (.not. comp(d)) then
        do m = 2*d, N, d
            comp(m) = .true.
        end do
        do m = d, N, d
            mu(m) = -mu(m)
        end do
        if (int(d,i8)*int(d,i8) <= int(N,i8)) then
            do m = d*d, N, d*d
                mu(m) = 0
            end do
        end if
    end if
end do
do d = 1, N
    do m = d, N, d
        dc(m) = dc(m) + 1
        acc(m) = acc(m) + mu(d)
    end do
end do
ok_odd = .true.; ok_bal = .true.; ok_prime = .true.; ok_seat = .true.
do m = 1, N
    r = int(sqrt(real(m, dp)))
    do while (r*r > m); r = r - 1; end do
    do while ((r+1)*(r+1) <= m); r = r + 1; end do
    nsq = merge(1, 0, r*r == m)
    if (mod(dc(m), 2) /= nsq) ok_odd = .false.
    if (m == 1 .and. acc(m) /= 1) ok_bal = .false.
    if (m > 1 .and. acc(m) /= 0) ok_bal = .false.
    if (m > 1 .and. ((dc(m) == 2) .neqv. (.not. comp(m)))) ok_prime = .false.
    if (m > 1 .and. .not. comp(m) .and. nsq /= 0) ok_seat = .false.
end do
call check('V: the fibre is odd exactly when it holds a seat, and holds at most one, n <= 10^6', ok_odd)
call check('V: a prime is exactly a fibre of one free orbit, with no seat, n <= 10^6', ok_prime .and. ok_seat)
call check('V: every finite fibre balances, the signs over the divisors sum to [n = 1], n <= 10^6', ok_bal)
ok_k4 = .true.
do k = 1, 6
    if (mod(kp(k), 420) /= mod(ks(k), 420)) ok_k4 = .false.
    if (dc(kp(k)) /= 2 .or. dc(ks(k)) /= 4) ok_k4 = .false.
    if (mu(kp(k)) /= -1 .or. mu(ks(k)) /= 1) ok_k4 = .false.
end do
call check('V: the parity frame: residues mod 420 merge each pair, the paid bits 1 and 2 separate it', ok_k4)
call check('V: thirty has the eight divisors of the sign cube, signs four and four', &
    dc(30) == 8 .and. sum(mu([1,2,3,5,6,10,15,30])) == 0)
deallocate(dc, acc, mu, comp)
end subroutine primes

! VI . wall, edge, seat
pure function theta(x) result(s)
real(dp), intent(in) :: x
real(dp) :: s
integer :: n
s = 0.0_dp
do n = 80, 1, -1
    s = s + exp(-acos(-1.0_dp) * real(n,dp)**2 * x)
end do
s = 1.0_dp + 2.0_dp * s
end function theta
subroutine wall_edge_seat()
integer, parameter :: NW = 10000000
real(dp), parameter :: xs(7) = [0.25_dp, 0.5_dp, 0.8_dp, 1.0_dp, 1.5_dp, 2.0_dp, 4.0_dp]
real(dp) :: worst, rr, s1, s2, h, best, ratio
integer(i8) :: n, lo, hi, mm

```

```

integer :: k, j, d, m, xbest
integer(1), allocatable :: mu(:)
logical, allocatable :: comp(:)
logical :: ok1, ok2, ok3
write(*,'(a)') 'VI WALL, EDGE, SEAT'
worst = 0.0_dp
do k = 1, 7
  rr = abs(theta(1.0_dp/xs(k)) - sqrt(xs(k))*theta(xs(k))) / (sqrt(xs(k))*theta(xs(k)))
  worst = max(worst, rr)
end do
write(*,'(a,e10.3)') ' theta(1/x) against sqrt(x) theta(x), worst relative gap over seven scales: ', worst
call check('VI: the lattice sum is self-dual; its weight is the square root, the fold's centre', worst < 1.0e-13_dp)
ok1 = .true.; ok2 = .true.; ok3 = .true.; h = 1.0_dp
do j = 0, 24
  lo = 2_i8**j + 1_i8; hi = 2_i8**(j+1)
  s1 = 0.0_dp; s2 = 0.0_dp
  do n = hi, lo, -1
    s1 = s1 + 1.0_dp/real(n,dp)
    s2 = s2 + 1.0_dp/real(n,dp)**2
  end do
  h = h + s1
  if (s1 < 0.5_dp .or. s1 > 1.0_dp) ok1 = .false.
  if (s2 > 2.0_dp**(-j)) ok2 = .false.
  if (h < 1.0_dp + 0.5_dp*real(j+1,dp)) ok3 = .false.
end do
call check('VI: every block of shares one-in-n holds between a half and one: the wall at Re s = 1', ok1 .and. ok3)
call check('VI: at two units per bit every block holds at most 2^-j: finite past the wall', ok2)
allocate(mu(NW), comp(NW))
mu = 1_1; comp = .false.
do d = 2, NW
  if (.not. comp(d)) then
    do m = 2*d, NW, d
      comp(m) = .true.
    end do
    do m = d, NW, d
      mu(m) = -mu(m)
    end do
    if (int(d,i8)*int(d,i8) <= int(NW,i8)) then
      do m = d*d, NW, d*d
        mu(m) = 0_1
      end do
    end if
  end if
end do
mm = 0_i8; best = 0.0_dp; xbest = 0
do m = 1, NW
  mm = mm + int(mu(m), i8)
  if (m >= 100) then
    ratio = abs(real(mm,dp)) / sqrt(real(m,dp))
    if (ratio > best) then
      best = ratio; xbest = m
    end if
  end if
end do
write(*,'(a,i0,a,f8.5,a,i0)') ' the signed walk of the paid parities to 10^7: M = ', mm, &
  '; max |M(x)|/sqrt(x) on [100, 10^7] = ', best, ' at x = ', xbest
call check('VI: the walk is paid to 10^7 and stays inside the square root there, a finite payment', best < 1.0_dp)
deallocate(mu, comp)
end subroutine wall_edge_seat

! VII . the heat flow: every zero moves by the pull law; the pair falls onto the line
subroutine flow_coeffs(c0, t, c)
  real(dp), intent(in) :: c0(0:6), t
  real(dp), intent(out) :: c(0:6)
  real(dp) :: dk(0:6), tmp(0:6), fac
  integer :: k, i
  c = c0; dk = c0; fac = 1.0_dp
  do k = 1, 3
    tmp = 0.0_dp
    do i = 2, 6
      tmp(i-2) = dk(i) * real(i,dp) * real(i-1,dp)
    end do
    dk = tmp
    fac = fac * (-t) / real(k,dp)
    c = c + fac * dk
  end do
end subroutine flow_coeffs
pure function peval(c, z) result(p)
  real(dp), intent(in) :: c(0:6)
  complex(dp), intent(in) :: z
  complex(dp) :: p
  integer :: i
  p = cmplx(c(6), 0.0_dp, dp)

```

```

do i = 5, 0, -1
  p = p*z + c(i)
end do
end function peval
pure function dpeval(c, z) result(p)
  real(dp), intent(in) :: c(0:6)
  complex(dp), intent(in) :: z
  complex(dp) :: p
  integer :: i
  p = cmplx(6.0_dp*c(6), 0.0_dp, dp)
  do i = 5, 1, -1
    p = p*z + real(i,dp)*c(i)
  end do
end function dpeval
subroutine newton_all(c, z)
  real(dp), intent(in) :: c(0:6)
  complex(dp), intent(inout) :: z(6)
  integer :: k, it
  do k = 1, 6
    do it = 1, 40
      z(k) = z(k) - peval(c, z(k)) / dpeval(c, z(k))
    end do
  end do
end subroutine newton_all
integer function sign_changes(c)
  real(dp), intent(in) :: c(0:6)
  integer :: i
  real(dp) :: x, v, vprev
  sign_changes = 0
  vprev = real(peval(c, cmplx(-6.0_dp, 0.0_dp, dp)), dp)
  do i = 1, 240000
    x = -6.0_dp + 12.0_dp*real(i,dp)/240000.0_dp
    v = real(peval(c, cmplx(x, 0.0_dp, dp)), dp)
    if (v*vprev < 0.0_dp) sign_changes = sign_changes + 1
    if (abs(v) > 0.0_dp) vprev = v
  end do
end function sign_changes
subroutine heat_flow()
  complex(dp) :: r0(6), zp(6), zm(6), z(6), poly(0:6), vf, vfd
  real(dp) :: c0(0:6), c(0:6), hh, worst, t, imprev
  integer :: k, j, i, nreal0, nreal1, step
  logical :: mono
  write(*,'(a)') 'VII THE HEAT FLOW'
  r0 = [cmplx(0.3_dp, 0.5_dp, dp), cmplx(0.3_dp, -0.5_dp, dp), cmplx(-2.0_dp, 0.0_dp, dp), &
        cmplx(-1.0_dp, 0.0_dp, dp), cmplx(1.5_dp, 0.0_dp, dp), cmplx(2.5_dp, 0.0_dp, dp)]
  poly = (0.0_dp, 0.0_dp); poly(0) = (1.0_dp, 0.0_dp)
  do k = 1, 6
    do i = k, 1, -1
      poly(i) = poly(i-1) - r0(k)*poly(i)
    end do
    poly(0) = -r0(k)*poly(0)
  end do
  c0 = real(poly, dp)
  hh = 1.0e-5_dp; worst = 0.0_dp
  call flow_coeffs(c0, hh, c); zp = r0; call newton_all(c, zp)
  call flow_coeffs(c0, -hh, c); zm = r0; call newton_all(c, zm)
  do k = 1, 6
    vf = (0.0_dp, 0.0_dp)
    do j = 1, 6
      if (j /= k) vf = vf + 2.0_dp / (r0(k) - r0(j))
    end do
    vfd = (zp(k) - zm(k)) / (2.0_dp*hh)
    worst = max(worst, abs(vfd - vf)/abs(vf))
  end do
  write(*,'(a,e10.3)') ' velocity of each zero against 2 sum 1/(z_k - z_j), worst relative gap: ', worst
  call check('VII: every zero moves by the pull law, degree six', worst < 1.0e-6_dp)
  z = r0; t = 0.0_dp; imprev = aimag(z(1)); mono = .true.; step = 0
  do while (aimag(z(1)) > 0.02_dp .and. step < 100000)
    step = step + 1; t = t + 1.0e-4_dp
    call flow_coeffs(c0, t, c); call newton_all(c, z)
    if (aimag(z(1)) >= imprev) mono = .false.
    imprev = aimag(z(1))
  end do
  write(*,'(a,f8.5)') ' the off-line pair reaches |Im| < 0.02 at t = ', t
  call check('VII: the pair falls toward the line at every step', mono .and. step < 100000)
  nreal0 = sign_changes(c0)
  call flow_coeffs(c0, 1.0_dp, c); nreal1 = sign_changes(c)
  write(*,'(a,i0,a,i0)') ' real zeros at t = 0: ', nreal0, ' at t = 1: ', nreal1
  call check('VII: four on the line before, six after: the pair crossed onto the seat', nreal0 == 4 .and. nreal1 == 6)
end subroutine heat_flow
end program bridge_twin

```

The manifest. MANIFEST.sha256

```

13476bea4c3851bb443fb449c6d9fe0ee3cacf9ba7762fec99ec8f2e19f9592 Bridge_From_First_Principle.lean
59ebdd19acfb90b762e5df1211513fc01b0d54f2568dfd6cde6044e780ca551a Bridge_Twin.f90
716c4f4613d17f1f94ebeca9aaf67e3862c1a491175ba3519c883a166764b394 bridge_check.py
07399a2e13d99c9ddc8db683aa1a76b71490349e37a4fbca92d6a410fae1460d bridge_boot.sh

```

APPENDIX B • THE RECEIPTS

Each receipt below is the output of the harness run on the files as extracted from this document. They are records, not warrants; the harness reproduces them.

B.1 • The boot, mode quick

```

BRIDGE FROM FIRST PRINCIPLE • RECEIPT • mode quick
TOOLCHAIN Lean (version 4.19.0, x86_64-unknown-linux-gnu, commit 6caae842e94, Release) • GNU Fortran (Ubuntu 13.3.0-6ubuntu2~24.04.1)
↳ 13.3.0
MANIFEST sha256 8d888c0180a6887b
GROUND toolchain present • manifest: 4 files signed, all verified • screen: no sorry, no axiom declared, no import, no bypass, no
↳ metaprogram, no IO
KERNEL exit 0 • 107 theorems, every cone pinned and printed by the compiler • 76 on no axiom • the rest within propext, Quot.sound,
↳ Classical.choice
CAPSTONE the_bridge_atom • no axiom
CAPSTONE the_bridge_from_first_principle • [propext, Classical.choice, Quot.sound]
TWIN built under -std=f2018 -O2 -fno-fast-math -ffp-contract=off -Wall -Wextra with zero warnings • 30 of 30 checks pass
CHAIN • D0 8d888c0180a6 -> D1 8f7b8ee44e84 -> D2 435ebd3e640b
BRIDGE EARNED • this run

```

B.2 • The judgment

```

BRIDGE FROM FIRST PRINCIPLE • JUDGMENT
NEGATION • 107 of 107 theorems refused their negation, each a proof failure
  instrument control: a planted vacuous law survived its negation, as dust must
INHABITATION • 15 conditional laws, each with a compiled model of its hypotheses, each model itself judged
JUDGMENT PASSED

```

B.3 • The controls

```

BRIDGE FROM FIRST PRINCIPLE • CONTROLS
CAUGHT manifest: an edited kernel
CAUGHT screen: a planted sorry
CAUGHT screen: a declared axiom
CAUGHT screen: an import
CAUGHT kernel: a drifted cone
CAUGHT kernel: the atom capstone negated
CAUGHT kernel: the last line altered
CAUGHT twin: a fused build (-ffp-contract=fast -march=native)
CAUGHT twin: the battery line altered
EARNED the clean copy earns
CONTROLS • 9 hostile copies, all refused; the clean copy earns

```

B.4 • The twin, its full run

```

0 CONDUCT
PASS 0: no fused multiply-add executed (a fused path reads -2^60)
PASS 0: NaN is ordered against nothing, itself included
I THE ATOM
PASS I: the fold undoes itself and fixes exactly the line, 1111 points
PASS I: the registration lands on the line, keeps the height, erases nothing exactly on it
PASS I: every off-line point has a distinct partner of one record, 1100 points
729 maps from records to points: returning both partners 0; returning one member 486
PASS I: no map from records returns both partners of a pair
PASS I: least erasure equals the value on all 4096 subsets of twelve points
II THREE, TWELVE, EIGHT, THE NINTH
PASS II: no two-axis system over GF(2) has exactly one solution, 256 systems
PASS II: 168 three-axis systems have determinant one and each locks all eight targets
PASS II: twelve rotations, and exactly one carries any directed gate to any directed gate
PASS II: eight patterns, total negation moves each and joins an even to an odd
PASS II: 2^k is never nine, k = 0..62
III PAY
PASS III: n distinct states fit injectively in m cells exactly when n <= m, every n, m <= 6
PASS III: the content halves and nothing merges exactly when the freedom at least doubles
PASS III: a merging step halves the content with no freedom exported (two states, one cell)
PASS III: the two-adic valuation is additive on positive arguments, 90000 pairs
IV SUPPLY
PASS IV: for every odd witness and odd target, exactly one calibration bit
PASS IV: of the 8 readings of a three-element content, none returns whether it ran
V PRIMES

```

PASS V: the fibre is odd exactly when it holds a seat, and holds at most one, $n \leq 10^6$
PASS V: a prime is exactly a fibre of one free orbit, with no seat, $n \leq 10^6$
PASS V: every finite fibre balances, the signs over the divisors sum to $[n = 1]$, $n \leq 10^6$
PASS V: the parity frame: residues mod 420 merge each pair, the paid bits 1 and 2 separate it
PASS V: thirty has the eight divisors of the sign cube, signs four and four
VI WALL, EDGE, SEAT
theta(1/x) against sqrt(x) theta(x), worst relative gap over seven scales: 2.212E-16
PASS VI: the lattice sum is self-dual; its weight is the square root, the fold's centre
PASS VI: every block of shares one-in-n holds between a half and one: the wall at $Re\ s = 1$
PASS VI: at two units per bit every block holds at most 2^{-j} : finite past the wall
the signed walk of the paid parities to 10^7 : $M = 1037$; max $|M(x)|/\sqrt{x}$ on $[100, 10^7] = 0.56710$ at $x = 199$
PASS VI: the walk is paid to 10^7 and stays inside the square root there, a finite payment
VII THE HEAT FLOW
velocity of each zero against $2 \sum 1/(z_k - z_j)$, worst relative gap: 1.009E-09
PASS VII: every zero moves by the pull law, degree six
the off-line pair reaches $|Im| < 0.02$ at $t = 0.09470$
PASS VII: the pair falls toward the line at every step
real zeros at $t = 0: 4$; at $t = 1: 6$
PASS VII: four on the line before, six after: the pair crossed onto the seat
BRIDGE-TWIN-JSON: {"checks":30,"failures":0}

B.5 • The cones, as the compiler prints them

The 107 lines below are the compiler's #print axioms output for every theorem of the kernel, the same lines the kernel pins.

'BFP.neg_neg_free' does not depend on any axioms
'BFP.neg_self_zero' does not depend on any axioms
'BFP.fold_involutive' does not depend on any axioms
'BFP.the_cut_is_the_line' does not depend on any axioms
'BFP.reg_lands' does not depend on any axioms
'BFP.reg_keeps_height' does not depend on any axioms
'BFP.reg_forgets_side' does not depend on any axioms
'BFP.reg_erases_nothing_iff' does not depend on any axioms
'BFP.off_line_pair' does not depend on any axioms
'BFP.one_bit_lost' does not depend on any axioms
'BFP.least_eraser_iff_value' does not depend on any axioms
'BFP.nothing_escapes_least_eraser' does not depend on any axioms
'BFP.pair_world_closed' does not depend on any axioms
'BFP.registered_closed' does not depend on any axioms
'BFP.one_record' does not depend on any axioms
'BFP.registered_has_value' does not depend on any axioms
'BFP.pair_world_lacks_value' does not depend on any axioms
'BFP.record_decides_nothing' does not depend on any axioms
'BFP.keyless_forces_nothing' does not depend on any axioms
'BFP.weakest_forcing_premise' does not depend on any axioms
'BFP.value_from_the_act' does not depend on any axioms
'BFP.reader_frame' does not depend on any axioms
'BFP.record_is_lossless' does not depend on any axioms
'BFP.record_same' does not depend on any axioms
'BFP.extremal_iff_value' does not depend on any axioms
'BFP.record_carried_uniquely' does not depend on any axioms
'BFP.chart_line_property' does not depend on any axioms
'BFP.cannot_lie' does not depend on any axioms
'BFP.cannot_deviate' does not depend on any axioms
'BFP.halted_iff' does not depend on any axioms
'BFP.cannot_reverse' does not depend on any axioms
'BFP.cannot_extend' depends on axioms: [Quot.sound]
'BFP.denial_is_coherent' does not depend on any axioms
'BFP.discriminator' depends on axioms: [propext, Classical.choice, Quot.sound]
'BFP.symmetry_is_keyless' does not depend on any axioms
'BFP.line_property_is_keyed' does not depend on any axioms
'BFP.socket_is_the_property' does not depend on any axioms
'BFP.no_socket_off_line' does not depend on any axioms
'BFP.act_seats_the_carrier' does not depend on any axioms
'BFP.two_axes_never_lock' does not depend on any axioms
'BFP.three_axes_lock' does not depend on any axioms
'BFP.third_begotten' does not depend on any axioms
'BFP.no_fourth_axis' does not depend on any axioms
'BFP.twelve_gates_one_torsor' does not depend on any axioms
'BFP.eight_patterns_four_orbits' does not depend on any axioms
'BFP.no_reflection_group_of_nine' depends on axioms: [propext]
'BFP.nine_is_eight_and_a_seat' does not depend on any axioms
'BFP.odd_fibre_iff_seat' does not depend on any axioms
'BFP.allLt_mem' does not depend on any axioms
'BFP.nd_map' depends on axioms: [propext, Quot.sound]
'BFP.allLt_map' does not depend on any axioms
'BFP.pigeonhole' depends on axioms: [propext, Quot.sound]
'BFP.export_doubles' depends on axioms: [propext, Quot.sound]
'BFP.one_bit_arrives' depends on axioms: [propext, Quot.sound]
'BFP.count_to_heat' depends on axioms: [propext, Quot.sound]

'BFP.merge_exports_nothing' does not depend on any axioms
 'BFP.fincfg_closed' does not depend on any axioms
 'BFP.least_eraser_iff_zero_erased' does not depend on any axioms
 'BFP.price_zero_iff_value' does not depend on any axioms
 'BFP.price_counts_pairs' does not depend on any axioms
 'BFP.additivity_over_zero_is_dry' depends on axioms: [propept, Quot.sound]
 'BFP.linear_on_tower' depends on axioms: [propept, Quot.sound]
 'BFP.bits_forced' depends on axioms: [propept, Quot.sound]
 'BFP.v2_mul_two' depends on axioms: [propept, Quot.sound]
 'BFP.v2_odd' depends on axioms: [propept, Quot.sound]
 'BFP.v2_two' depends on axioms: [propept, Quot.sound]
 'BFP.v2_split' depends on axioms: [propept, Quot.sound]
 'BFP.v2_pow_odd' depends on axioms: [propept, Quot.sound]
 'BFP.v2_additive' depends on axioms: [propept, Quot.sound]
 'BFP.wall' does not depend on any axioms
 'BFP.calibration' does not depend on any axioms
 'BFP.formal_never_odd' does not depend on any axioms
 'BFP.constant_no_crossing' does not depend on any axioms
 'BFP.no_reading_returns_the_deed' does not depend on any axioms
 'BFP.three_legs_one_orbit' does not depend on any axioms
 'BFP.prime_iff_one_free_orbit' does not depend on any axioms
 'BFP.parity_frame_paid_bits' does not depend on any axioms
 'BFP.divisor_cube_is_sign_cube' does not depend on any axioms
 'BFP.fibre_balance' does not depend on any axioms
 'BFP.wall_and_edge_exchanged' does not depend on any axioms
 'BFP.prime_mode_unitary_iff' depends on axioms: [propept, Quot.sound]
 'BFP.price_per_bit_tower' depends on axioms: [propept]
 'BFP.len_le_sum' depends on axioms: [propept, Quot.sound]
 'BFP.wall_block_share' depends on axioms: [propept, Quot.sound]
 'BFP.conjugation_is_the_fold' depends on axioms: [propept]
 'BFP.flow_inverse' depends on axioms: [propept, Quot.sound]
 'BFP.flow_injective' depends on axioms: [propept, Quot.sound]
 'BFP.disc_flow' depends on axioms: [propept, Quot.sound]
 'BFP.pull_in_square' depends on axioms: [propept, Quot.sound]
 'BFP.collision_sign' depends on axioms: [propept, Quot.sound]
 'BFP.zero_slack' depends on axioms: [propept, Quot.sound]
 'BFP.certificate_merges' does not depend on any axioms
 'BFP.charge_carrier' does not depend on any axioms
 'BFP.value_from_the_act_inhabited' does not depend on any axioms
 'BFP.off_line_inhabited' does not depend on any axioms
 'BFP.weakest_forcing_premise_inhabited' does not depend on any axioms
 'BFP.keyless_forces_nothing_inhabited' does not depend on any axioms
 'BFP.count_to_heat_inhabited' depends on axioms: [propept, Quot.sound]
 'BFP.bits_forced_inhabited' depends on axioms: [propept, Quot.sound]
 'BFP.calibration_inhabited' does not depend on any axioms
 'BFP.three_legs_inhabited' does not depend on any axioms
 'BFP.prime_mode_unitary_inhabited' does not depend on any axioms
 'BFP.zero_slack_inhabited' does not depend on any axioms
 'BFP.record_carried_inhabited' does not depend on any axioms
 'BFP.price_zero_inhabited' does not depend on any axioms
 'BFP.the_bridge_atom' does not depend on any axioms
 'BFP.the_bridge_from_first_principle' depends on axioms: [propept, Classical.choice, Quot.sound]